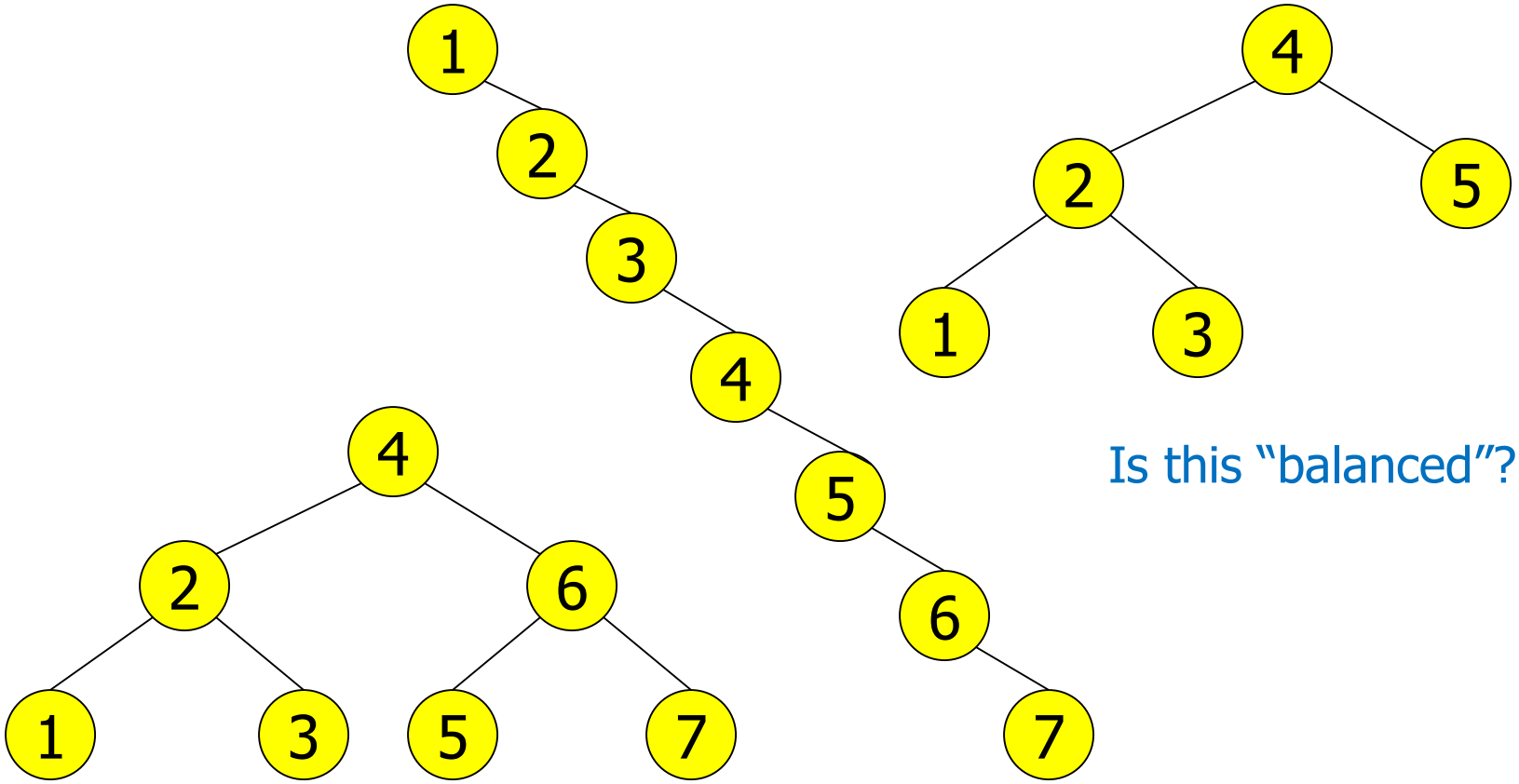
Data Structures

13. AVL Trees

Balanced and Unbalanced BST

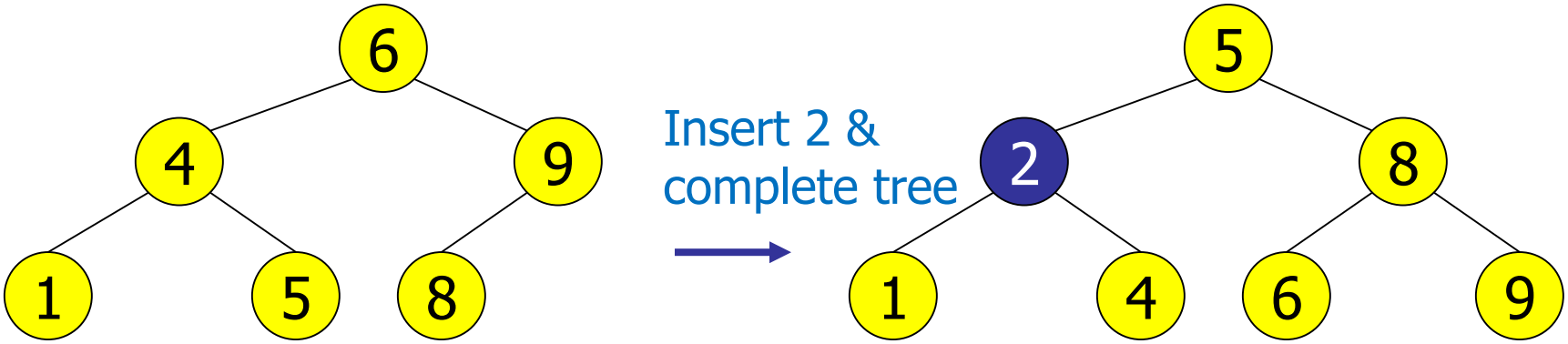


Is this “balanced”?

Balanced Tree

- Want a (almost) complete tree after every operation
  - Tree is complete except possibly in the lower levels towards right

- Maintenance of such a tree is expensive
  - For example, insert 2 in the tree on the left and then rebuild as a complete tree



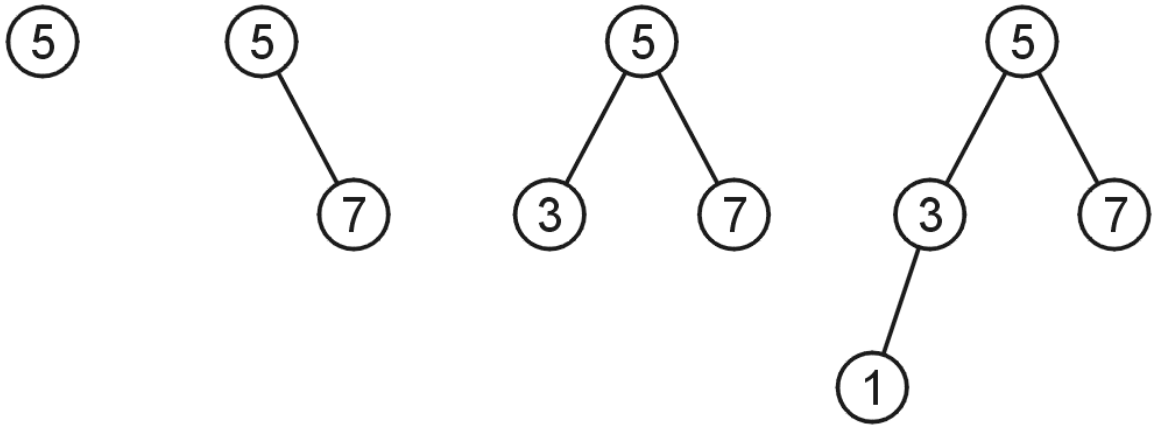
AVL Trees – Good but not Perfect Balance

- Named after Adelson-Velskii and Landis
- Balance is defined by comparing the height of the two sub-trees
- Recall:
  - An empty tree has height –1
  - A tree with a single node has height 0

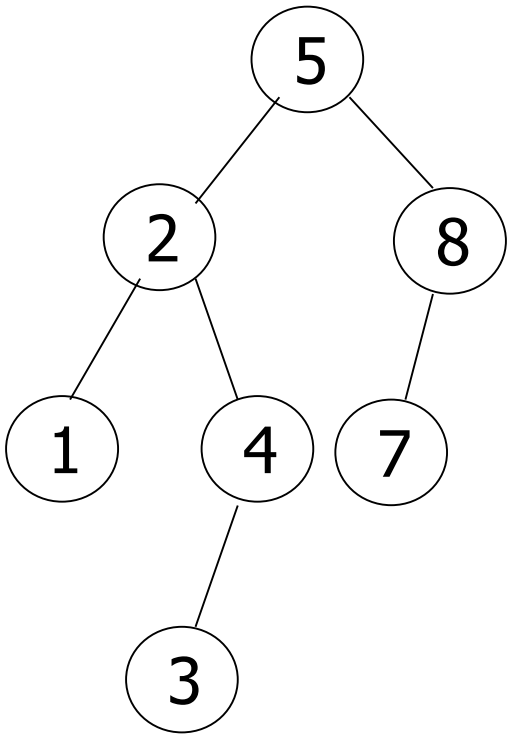
AVL Trees

- A **binary search tree** is said to be AVL **balanced** if:
  - The difference in the heights between the left and right sub-trees is at most 1, and
  - Both sub-trees are themselves AVL trees

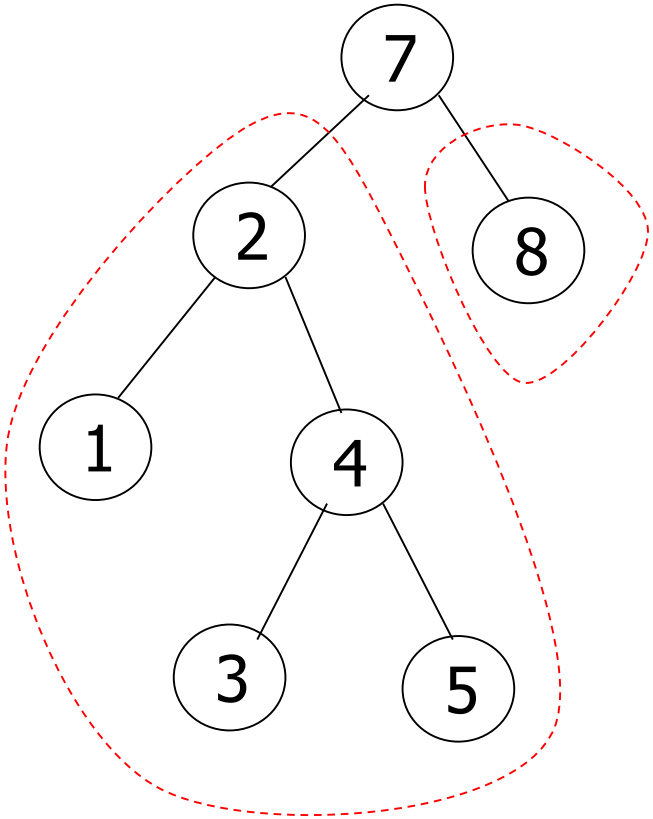
- AVL trees with 1, 2, 3 and 4 nodes



AVL Trees – Example



An AVL Tree



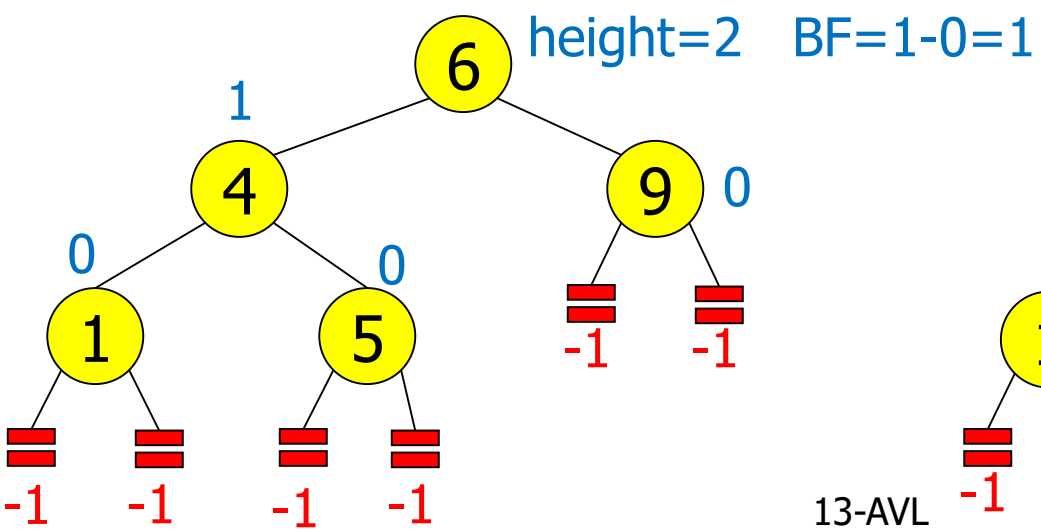
Not an AVL Tree

AVL Trees – Balance Factor

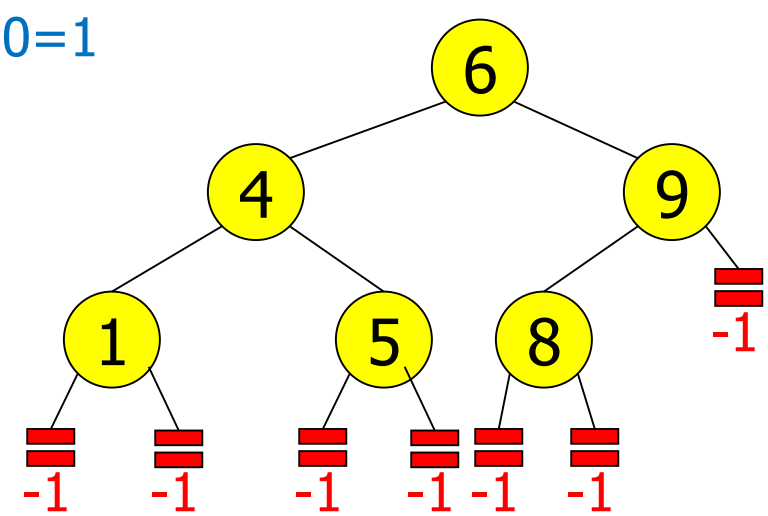
- An AVL tree has balance factor calculated at every node
  - Height of the left subtree minus the height of the right subtree
  - For an AVL tree, the balances of the nodes are always -1, 0 or 1

Height of node	= h
Balance Factor (BF)	= $h_{left} - h_{right}$
Empty height	= -1

Tree A (AVL)

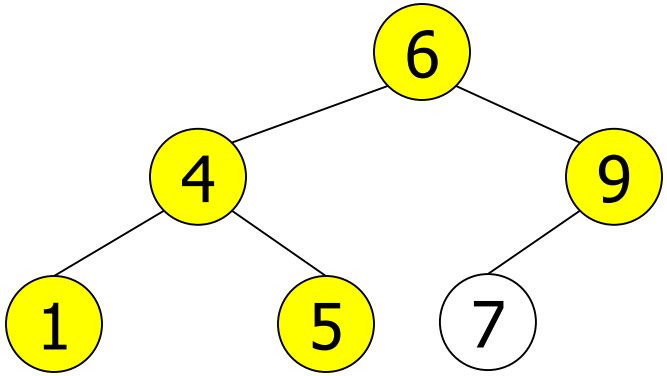


Tree B (AVL)

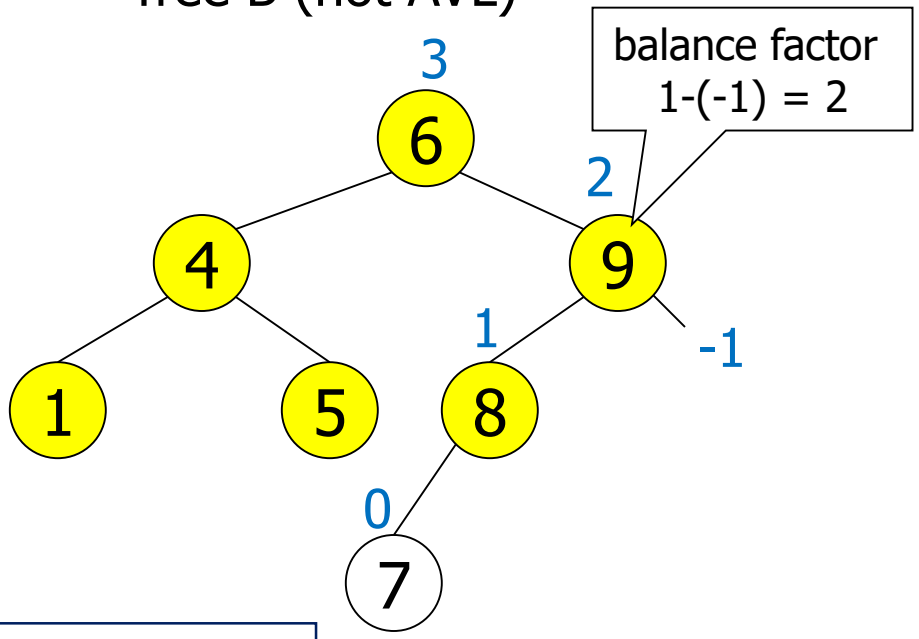


AVL Trees – Example

Tree A (AVL)



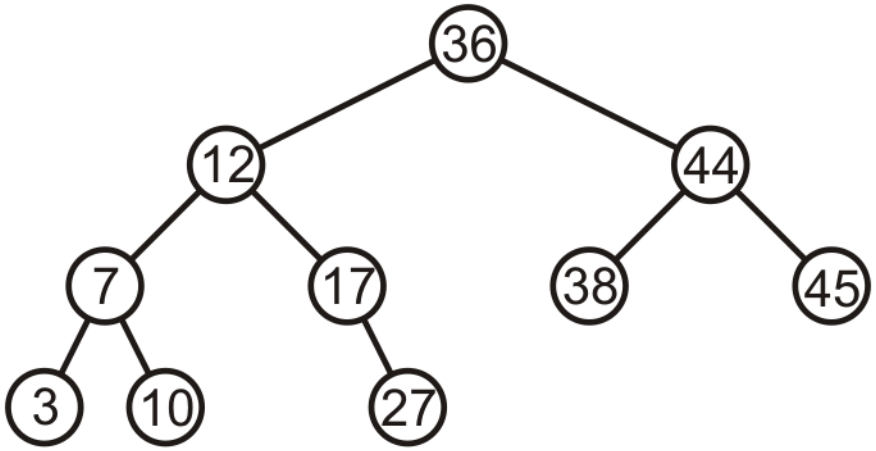
Tree B (not AVL)



Height of node = h
Balance factor = $h_{left}-h_{right}$
Empty height = -1

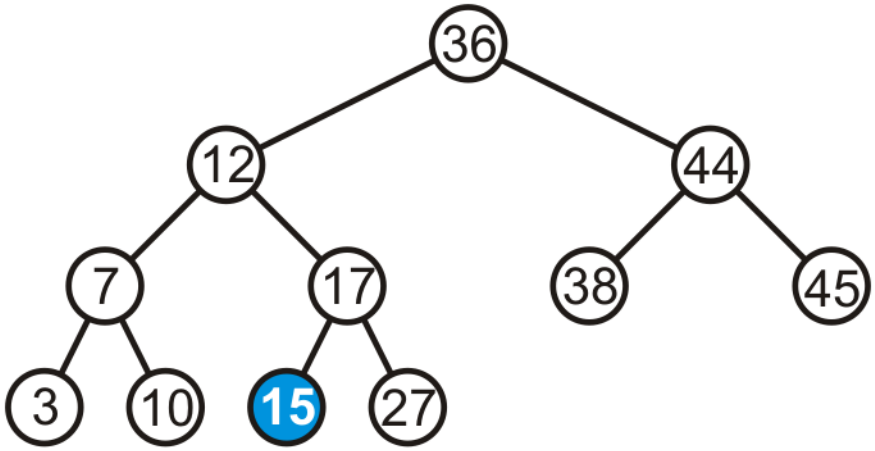
AVL Trees – Example

- Consider this AVL tree



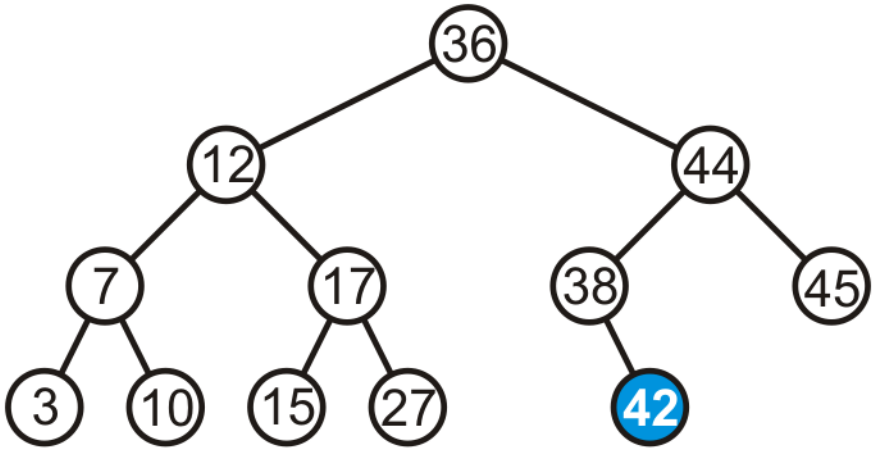
AVL Trees – Example

- Consider inserting 15 into this tree
  - In this case, the heights of none of the trees change
  - Tree remains balanced



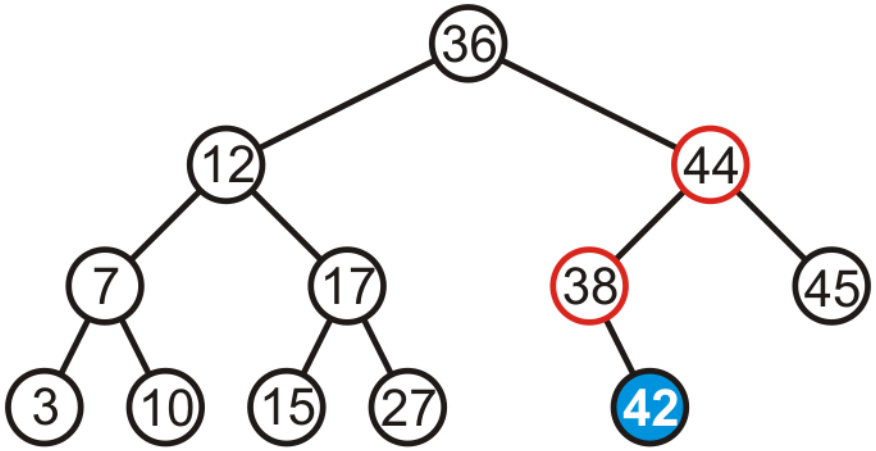
AVL Trees – Example

- Consider inserting 42 into this tree



AVL Trees – Example

- Consider inserting 42 into this tree
  - Height of two sub-trees rooted at 44 and 38 have increased by one
  - The tree is still balanced



AVL Trees

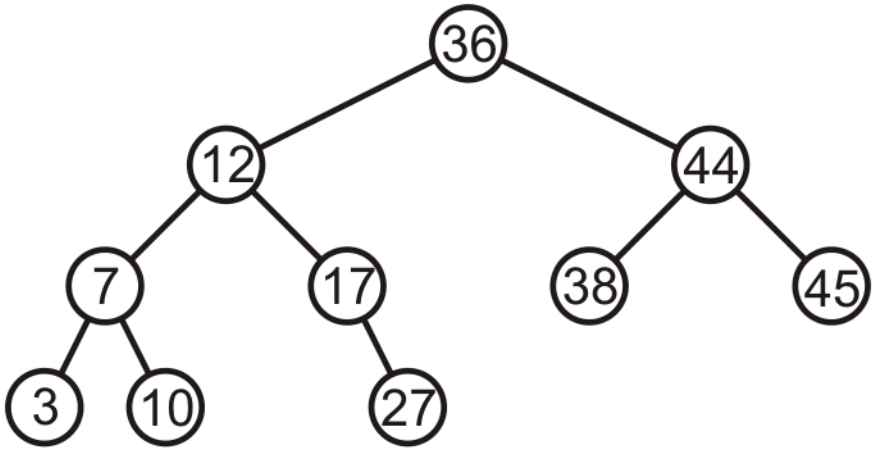
- To maintain the height balanced property of the AVL tree, it is necessary to
  - Perform a transformation on the tree, such that
  - In-order traversal of the transformed tree is the same as for the original tree (i.e., the new tree remains a binary search tree)

Transformation (Rotation) of AVL Trees

- Insert operations may cause balance factor to become 2 or –2 for some node
- Only nodes on the path from insertion point to root node have possibly change in height
- Follow the path up to the root, find the first node (i.e., deepest) whose new balance violates the AVL condition
  - Call this node "a"
- If a new balance factor (the difference h_{left}-h_{right}) is 2 or –2
  - Adjust tree by rotation around the node "a"

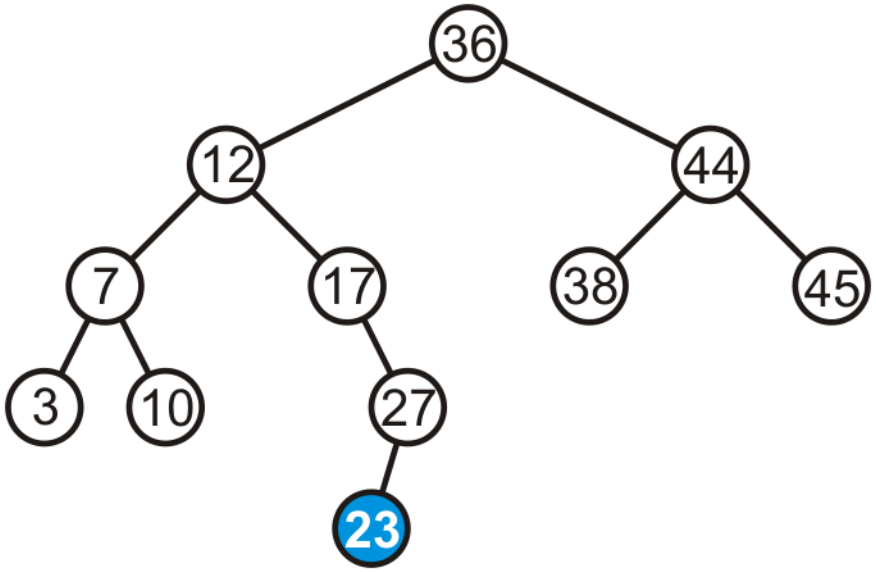
Balancing AVL Trees – Example

- If a tree is AVL balanced, for an insertion to cause an imbalance:
  - The heights of the sub-trees must differ by 1
  - The insertion must increase the height of the deeper sub-tree by 1



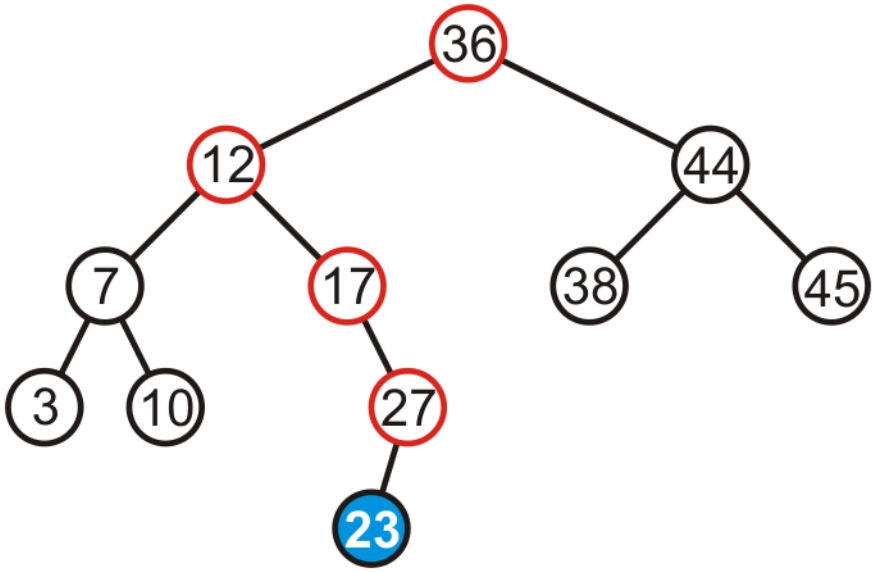
Balancing AVL Trees – Example

- Suppose we insert 23 into our initial tree



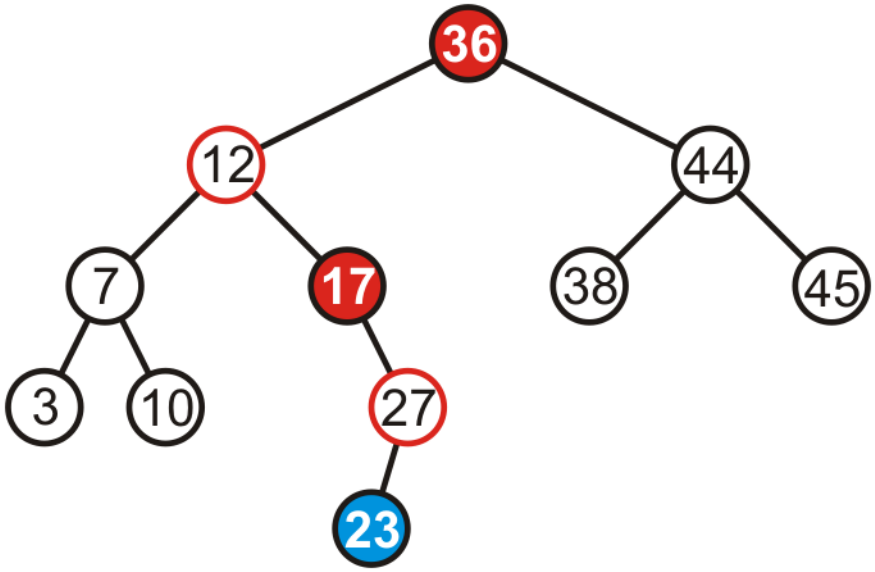
Balancing AVL Trees – Example

- The heights of each of the sub-trees from the insertion point to the root are increased by one



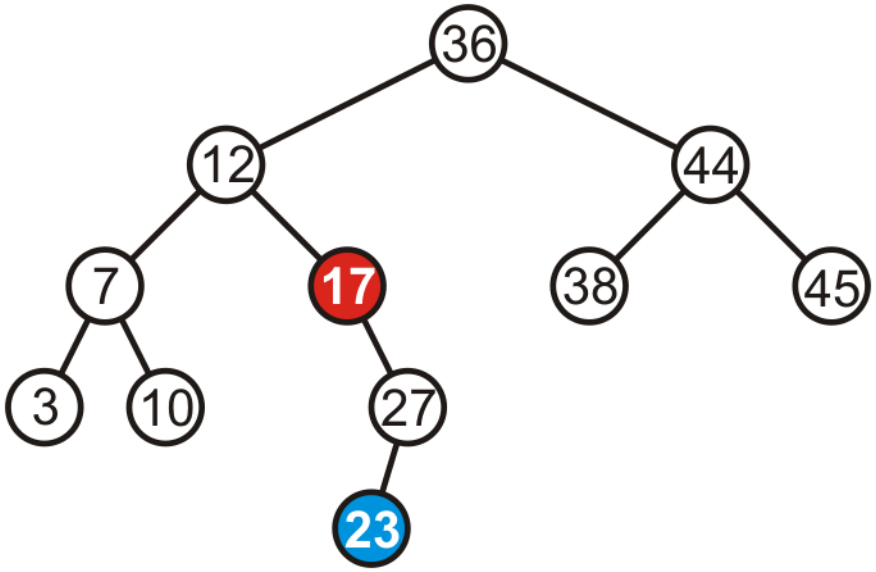
Balancing AVL Trees – Example

- Only two of the nodes are unbalanced, i.e., 17 and 36
  - Balance factor of 17 is -2
  - Balance factor of 36 is 2



Balancing AVL Trees – Example

- We only have to fix the imbalance at the lowest node



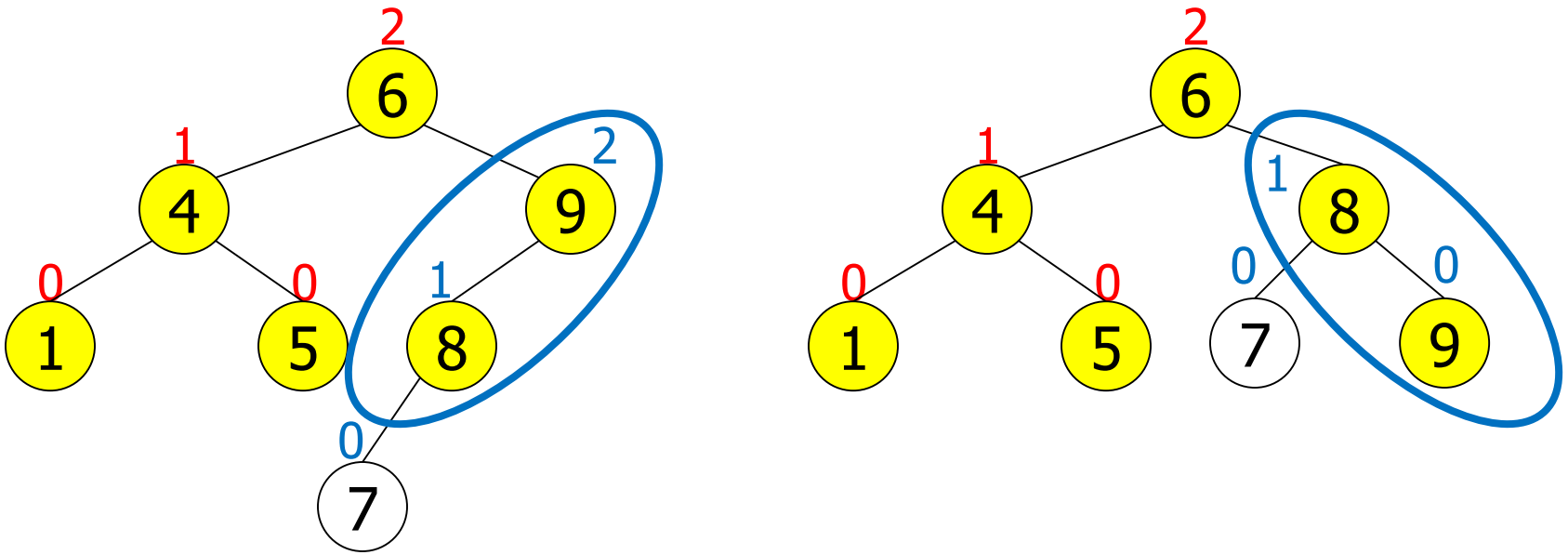
Fixing Imbalance By Rotation

- Let the node that needs rebalancing be "a"
- Imbalance during insertion may be handled using four cases

- Outside cases (Single Rotation)
  1. Right rotation (case RR)
  2. Left rotation (case LL)

- Inside cases (Double Rotation)
  3. Right-left rotation (case RL)
  4. Left-right rotation (case LR)

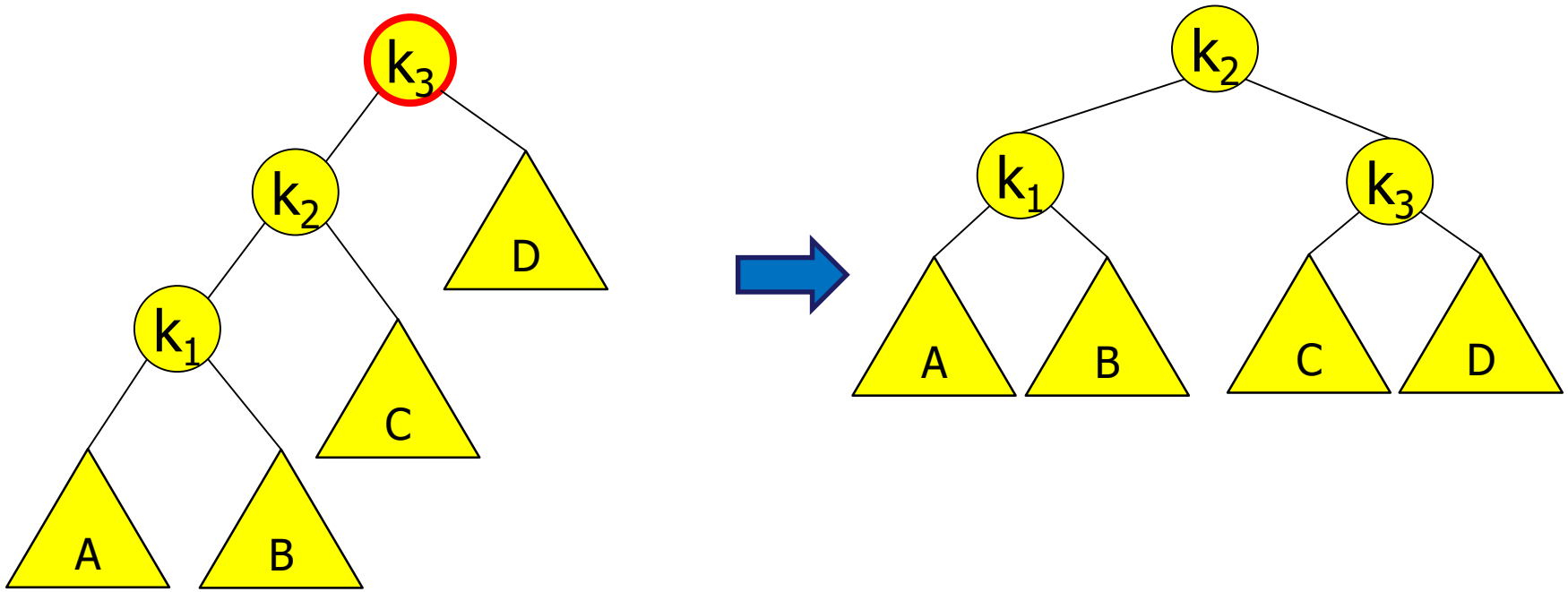
Single Rotation in an AVL Tree



Right Rotation (RR) in an AVL Tree

- Node “b” becomes the new root
- Node “b” takes ownership of node “a”, as it's right child
- Node “a” takes ownership of node “b” right child (NULL if no child)
  - As left child of node “a”

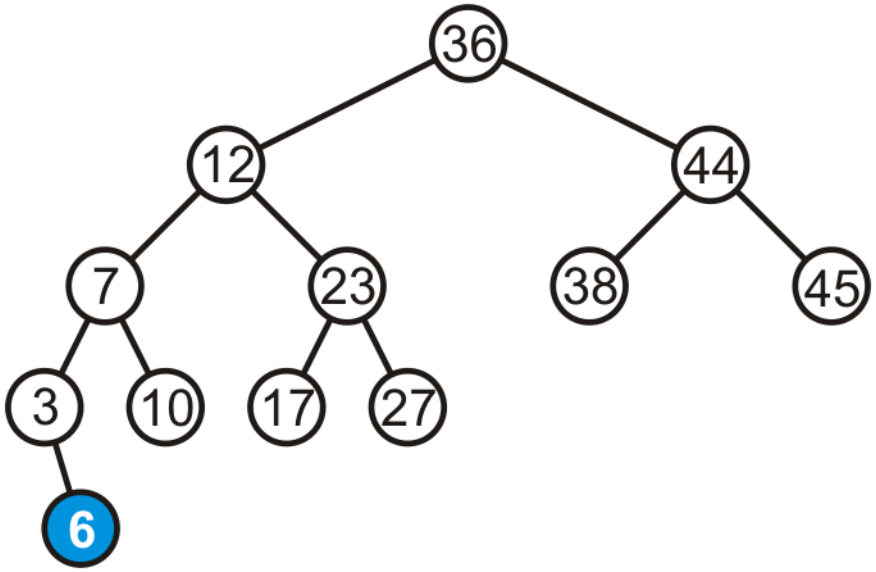
Right Rotation (RR) in an AVL Tree



- Node $\mathbf{k_2}$ becomes the new root
- Node $\mathbf{k_2}$ takes ownership of node $\mathbf{k_3}$, as its right child
- Node $\mathbf{k_3}$ takes ownership of node $\mathbf{k_2}$'s right child, as its left child

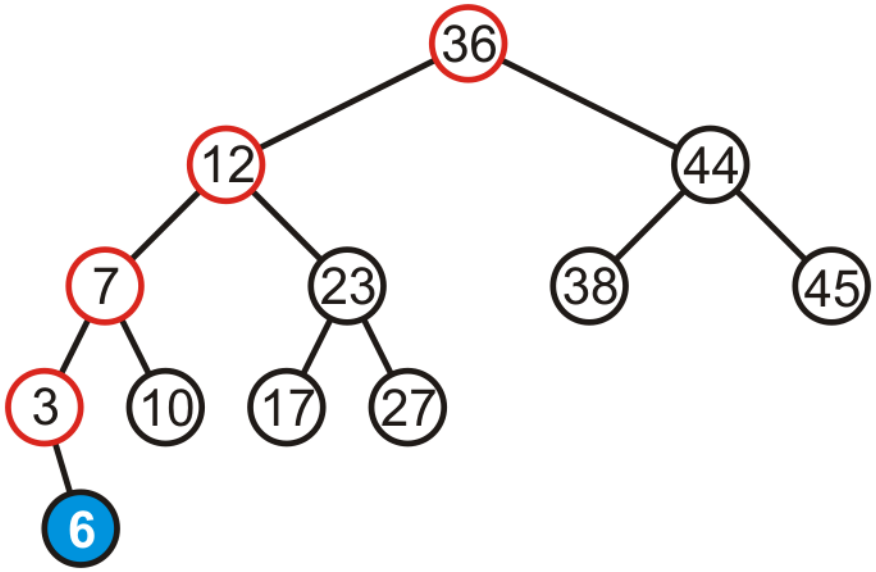
Right Rotation (RR) – Example

- Consider adding 6



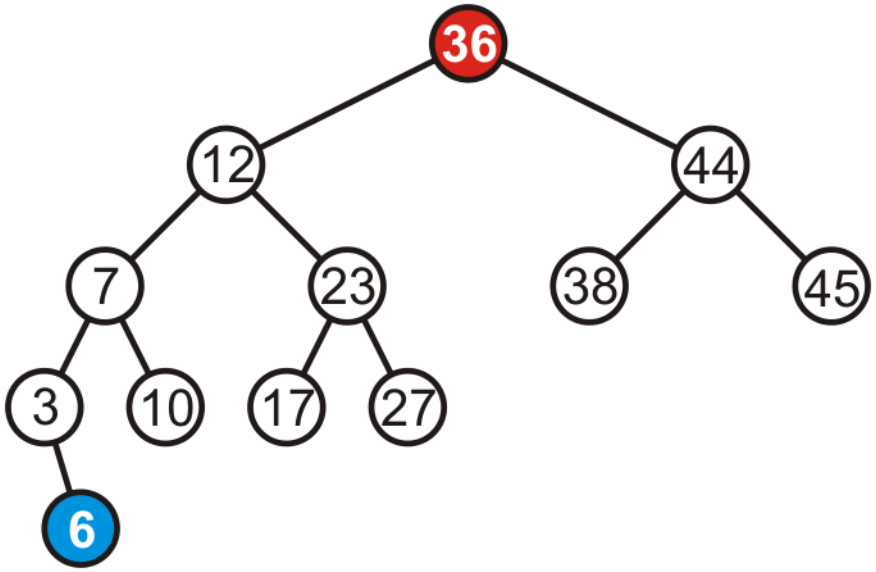
Right Rotation (RR) – Example

- Height of each of the trees in the path back to the root are increased by one



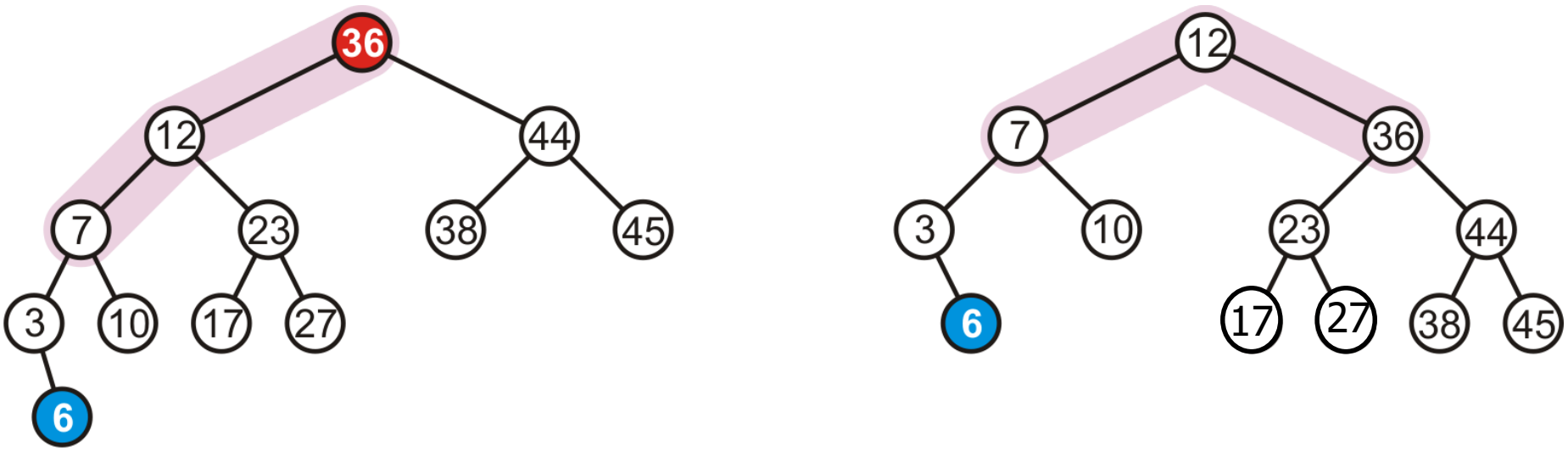
Right Rotation (RR) – Example

- Height of each of the trees in the path back to the root are increased by one
  - Only root node (i.e., 36) violates the balancing factor



Right Rotation (RR) – Example

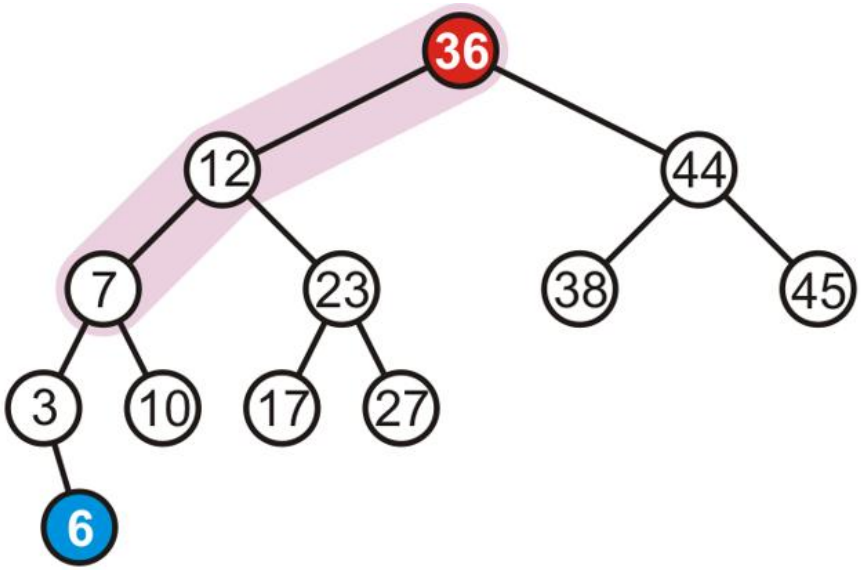
- To fix the imbalance, we perform right rotation of root (i.e., 36)



When to Perform Right Rotation (RR)?

- Let the node that needs rebalancing be "a"

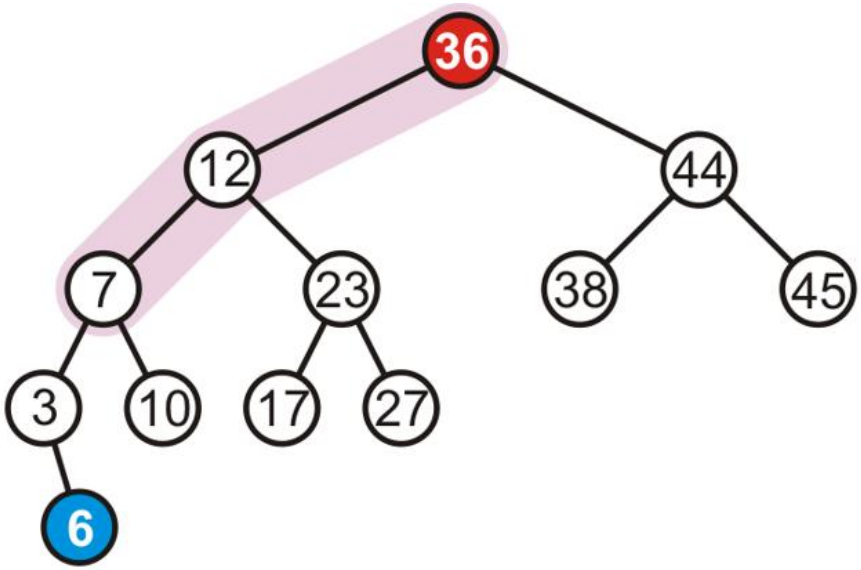
- Case RR
  - Insertion into left subtree of left child of node "a"
  - Left tree is heavy (i.e., $h_{left} > h_{right}$)



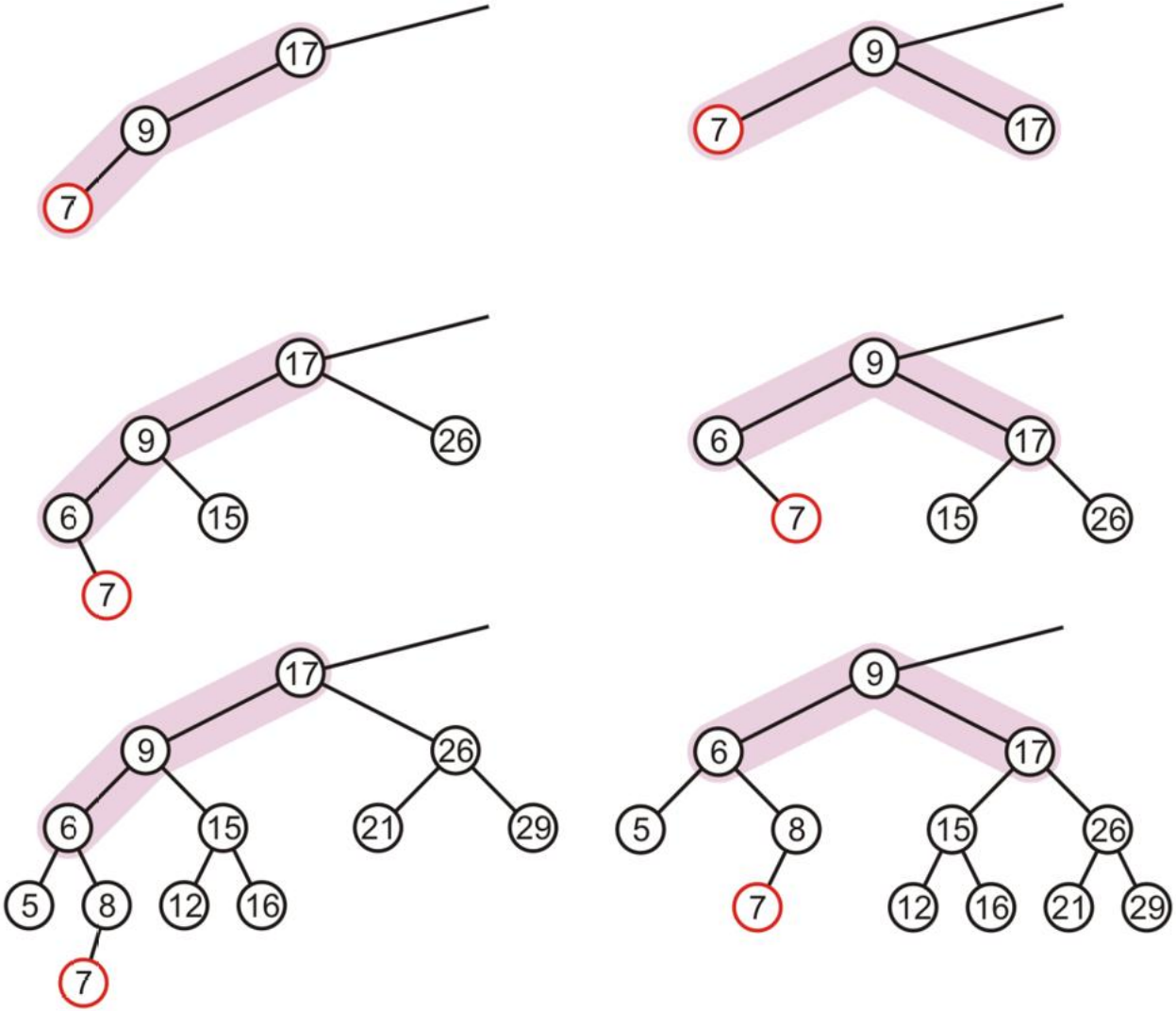
When to Perform Right Rotation (RR)

- Let the node that needs rebalancing be a

- Case RR
  - Insertion into left subtree of left child of node a (**RR**)
  - Left tree is heavy (i.e., $h_{left} > h_{right}$)



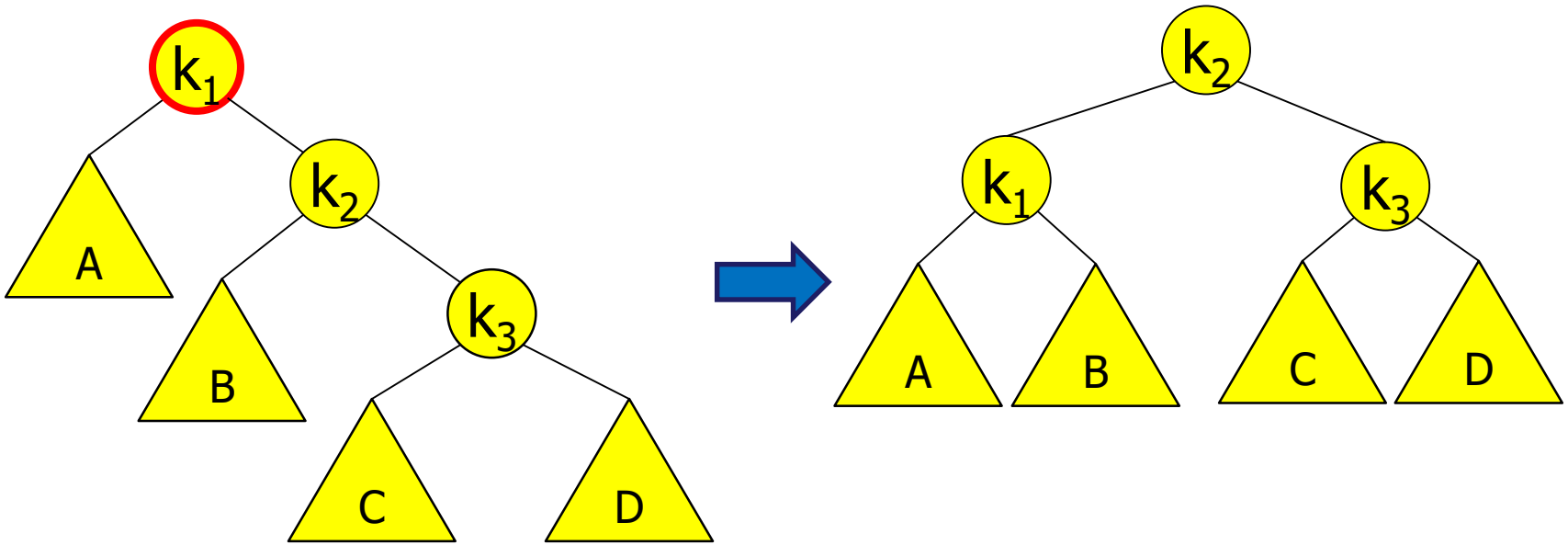
Right Rotation (RR) – Examples



Left Rotation (LL) in an AVL Tree

- Node "b" becomes the new root
- Node "b" takes ownership of node "a" as its left child
- Node "a" takes ownership of node "b" left child (NULL if no child)
  - As right child of node "a"

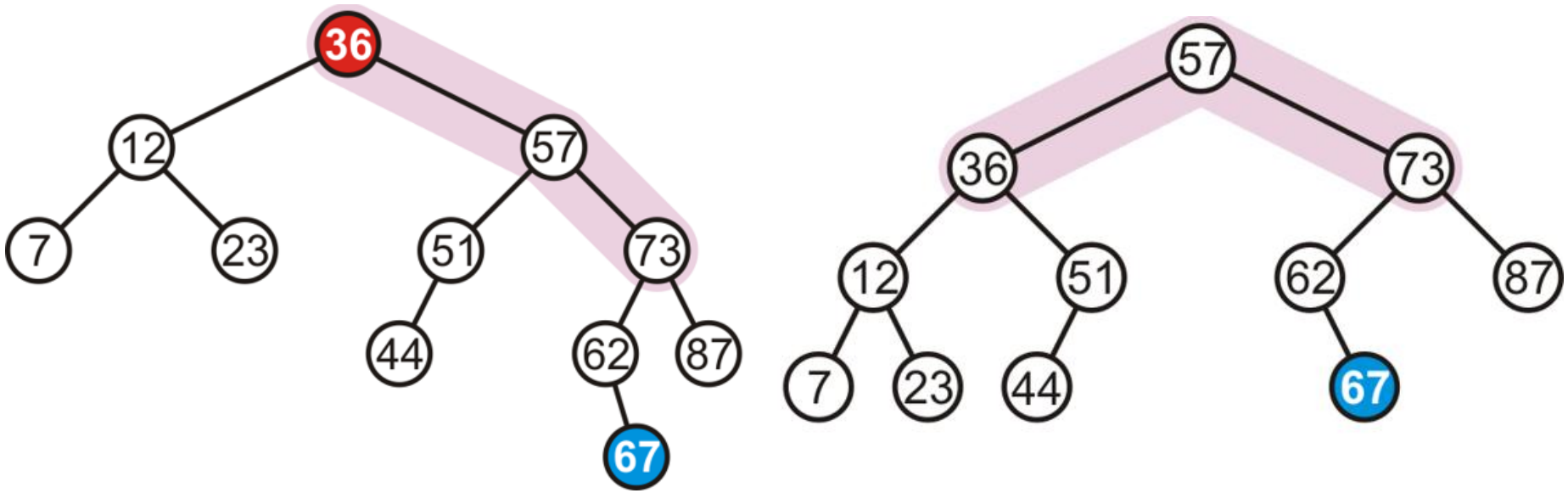
Left Rotation (LL) in an AVL Tree



- Node $\mathbf{k_2}$ becomes the new root
- Node $\mathbf{k_2}$ takes ownership of node $\mathbf{k_1}$ as its left child
- Node $\mathbf{k_1}$ takes ownership of node $\mathbf{k_2}$'s left child as its right child

Left Rotation (LL) – Example

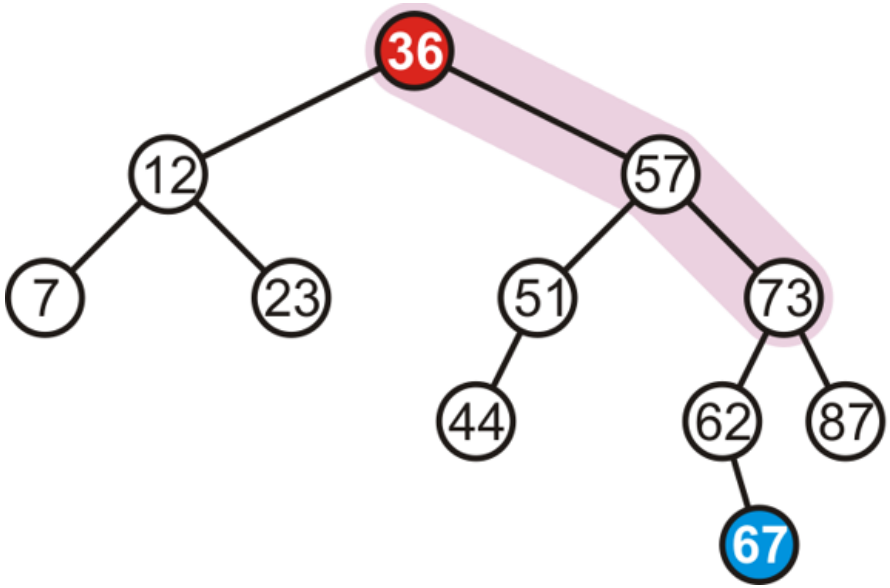
- Consider adding 67
  - To fix the imbalance, we perform left rotation of root (i.e., 36)



When to Perform Left Rotation (LL)

- Let the node that needs rebalancing be "a"

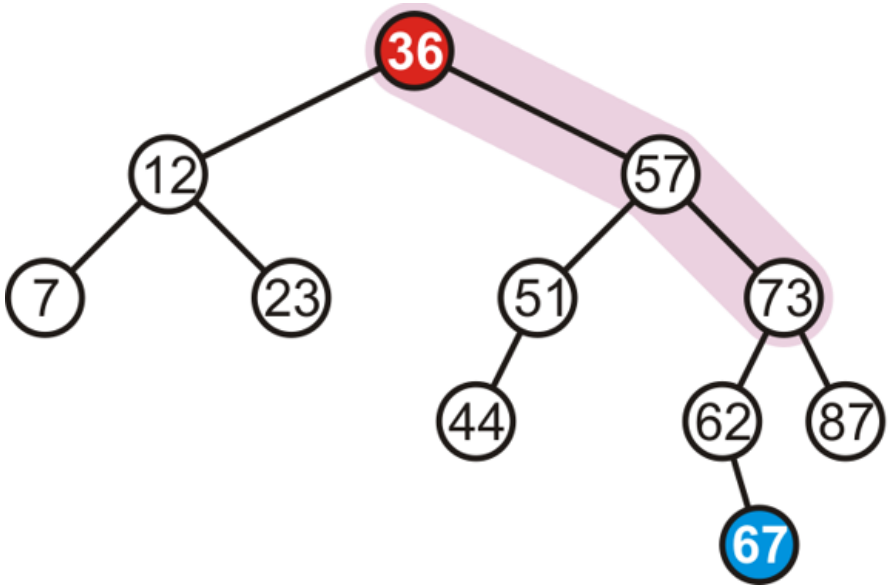
- Case LL
  - Insertion into right subtree of right child of node a
  - Right tree is heavy (i.e., $h_{left} < h_{right}$)



When to Perform Left Rotation (LL)

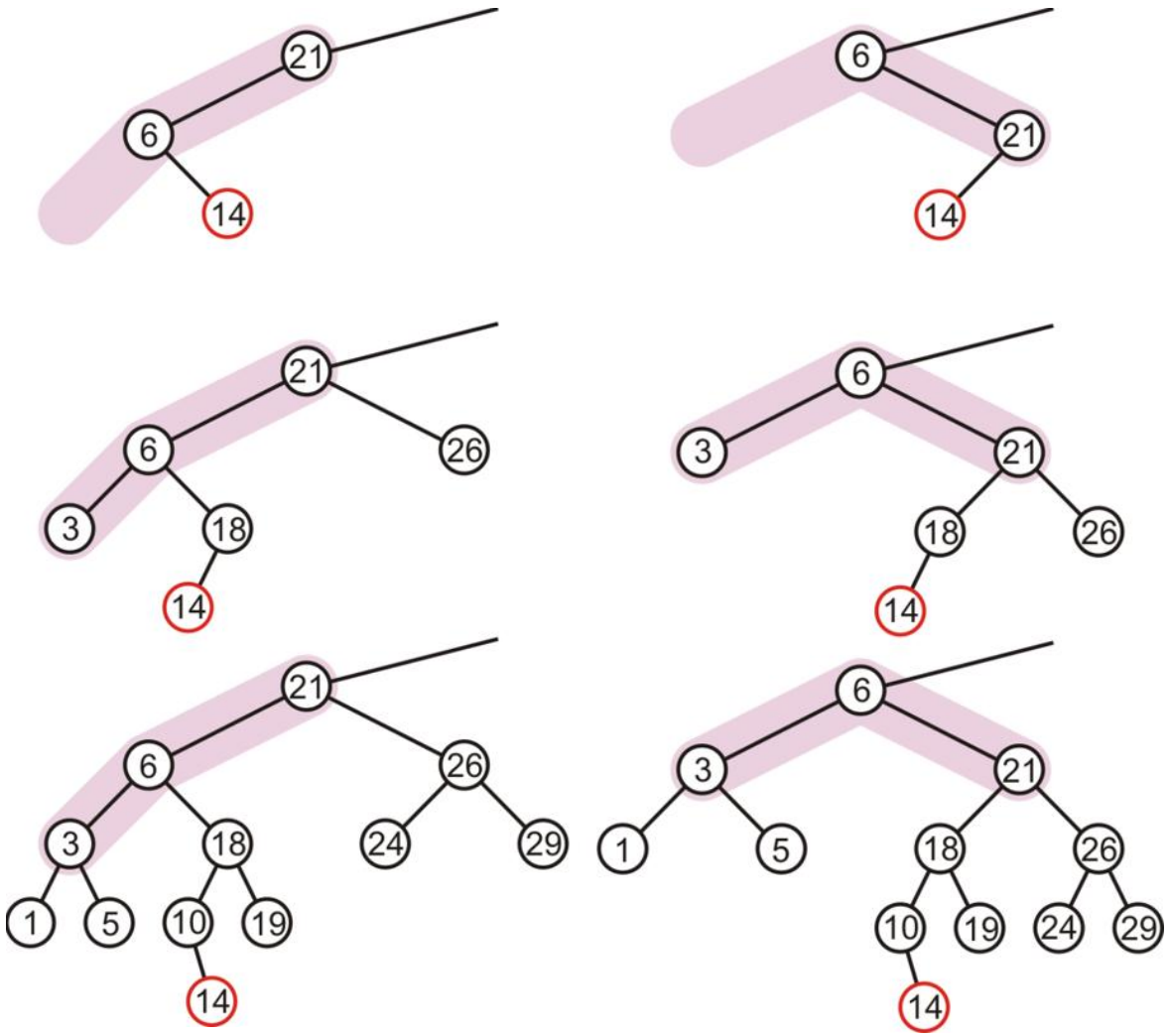
- Let the node that needs rebalancing be a

- Case LL
  - Insertion into right subtree of right child of node a (**LL**)
  - Right tree is heavy (i.e., $h_{left} < h_{right}$)

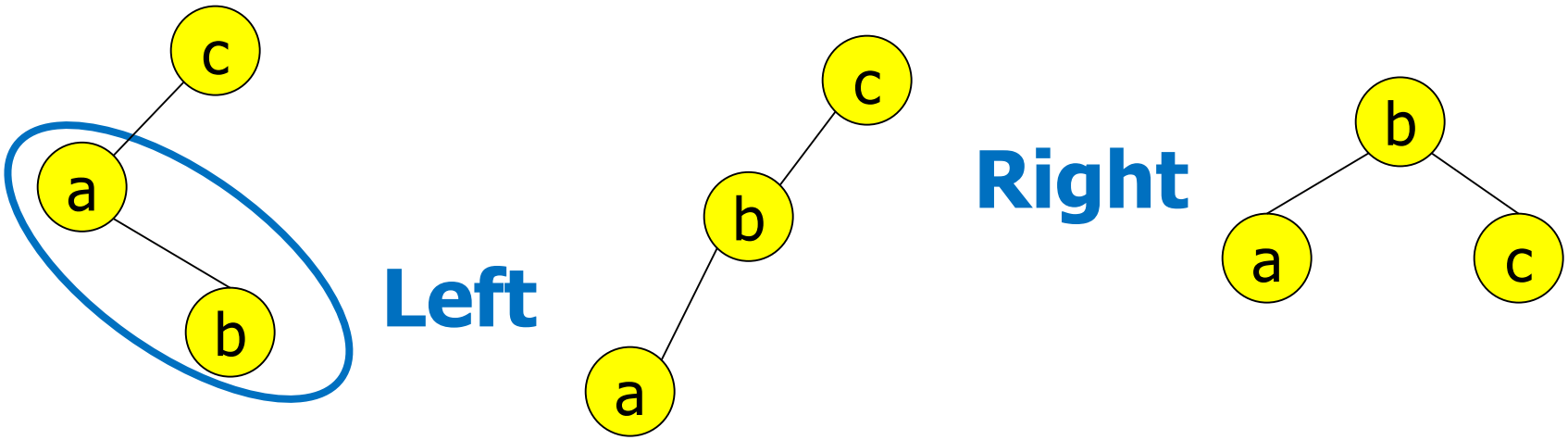


Single Rotation May Be Insufficient

- The imbalance is just shifted to the other side



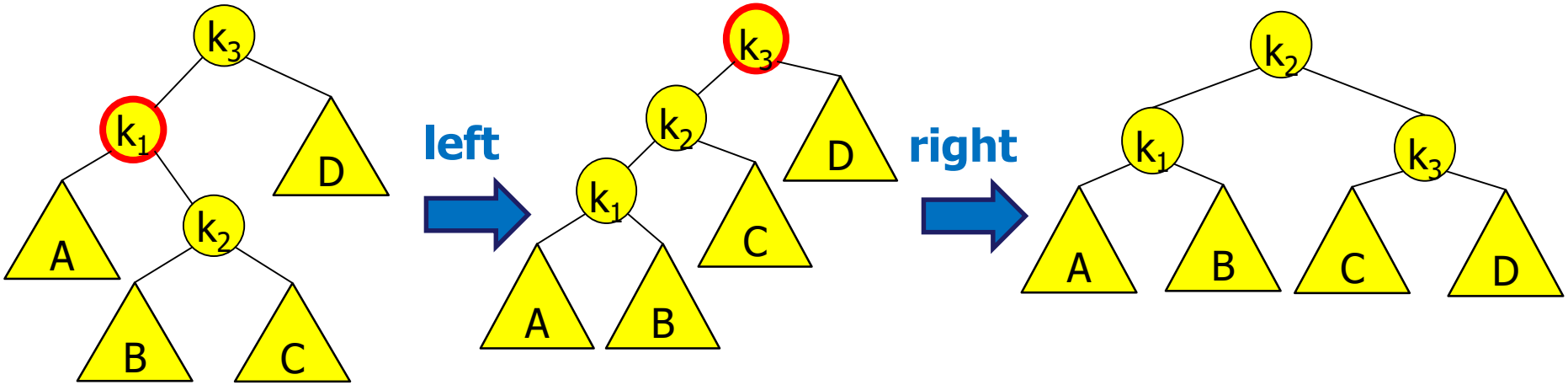
Left-Right Rotation (LR) or "Double Right"



- Perform a left rotation on the left subtree

- Node “b” becomes the new root
- Node “b” takes ownership of node “c” as its right child
- Node “c” takes ownership of node “b” right child
  - As its left child

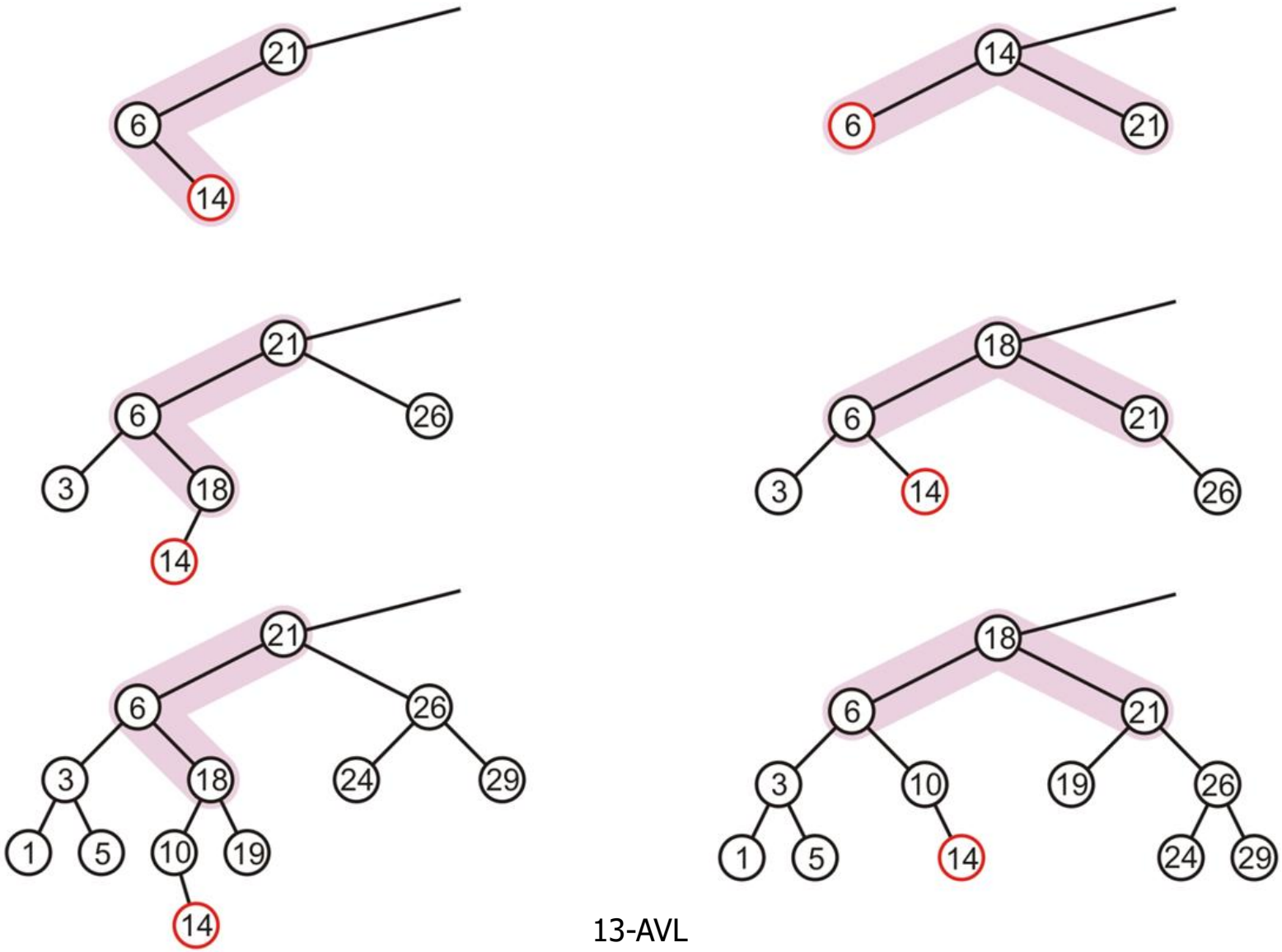
Left-Right Rotation (LR)



1. Perform a left rotation on the left subtree
2. Perform a right rotation on root

- Node $\mathbf{k_2}$ becomes the new root
- Node $\mathbf{k_2}$ takes ownership of node $\mathbf{k_3}$ as its right child
- Node $\mathbf{k_1}$ takes ownership of node $\mathbf{k_2}$'s left child as its right child
- Node $\mathbf{k_3}$ takes ownership of node $\mathbf{k_2}$'s right child as its left child

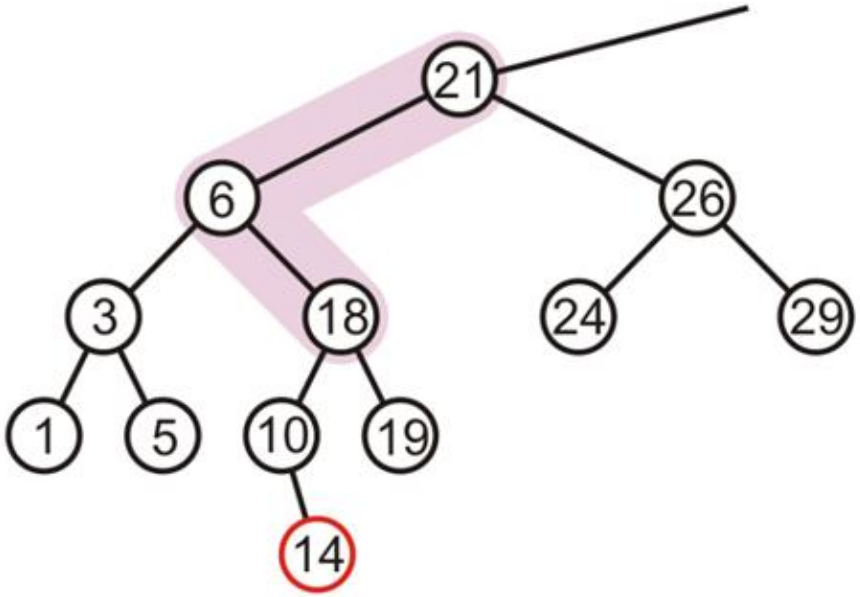
Left-Right Rotation (LR) or "Double Right" – Example



When to Perform Left-Right Rotation (LR)

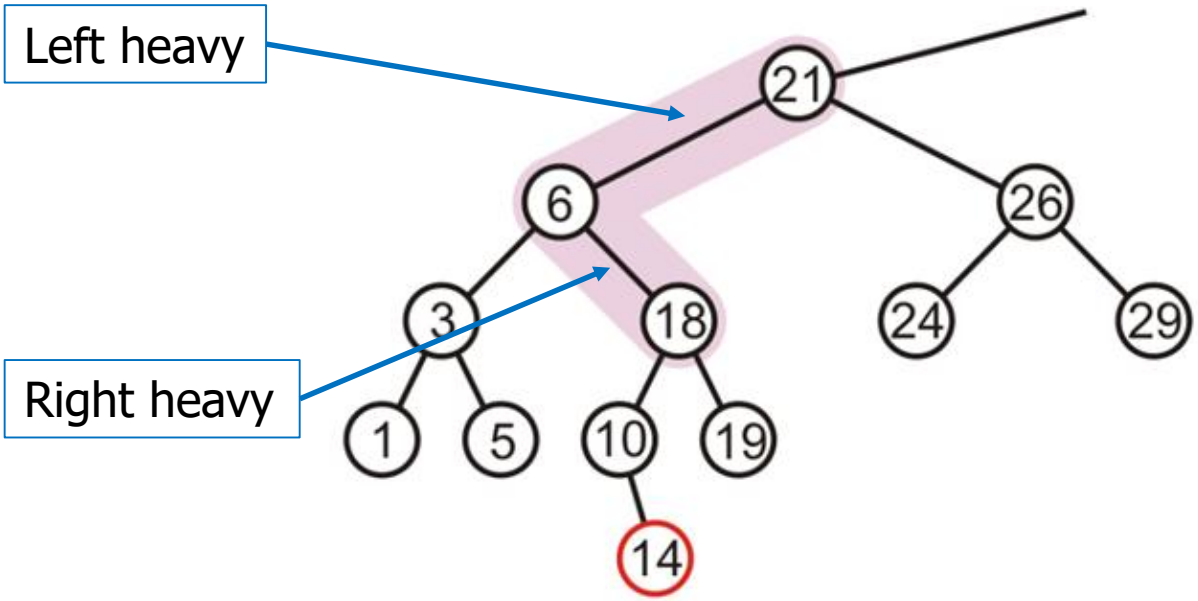
- Let the node that needs rebalancing be "a"

- Case RL
  - Insertion into right subtree of left child of node "a"

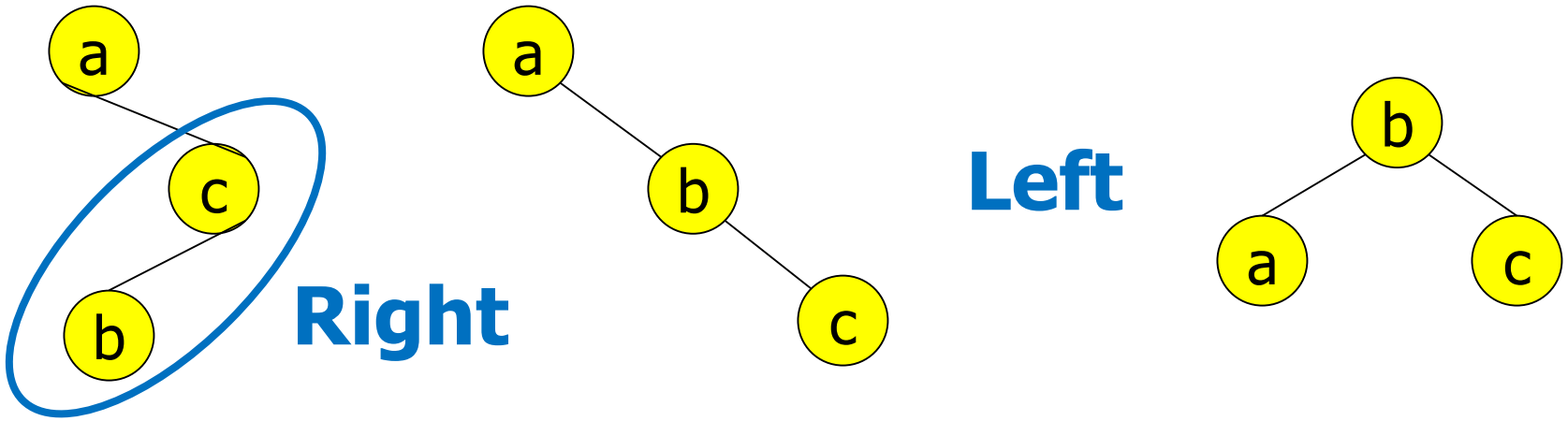


When to Perform Left-Right Rotation (LR)

- Let the node that needs rebalancing be a
- Case RL
  - Insertion into right subtree of left child of node a (**RL**)



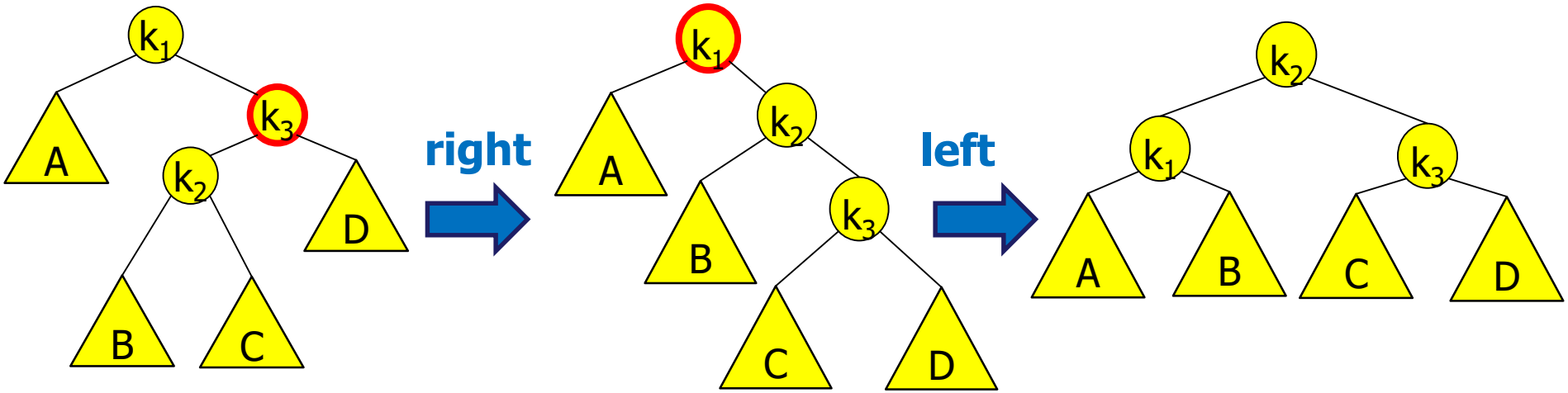
Right-Left Rotation (RL) or "Double Left"



- Perform a right rotation on the right subtree

- Node “b” becomes the new root
- Node “b” takes ownership of node “a” as its left child
- Node “a” takes ownership of node “b” left child
  - As its right child

Right-Left Rotation (RL)

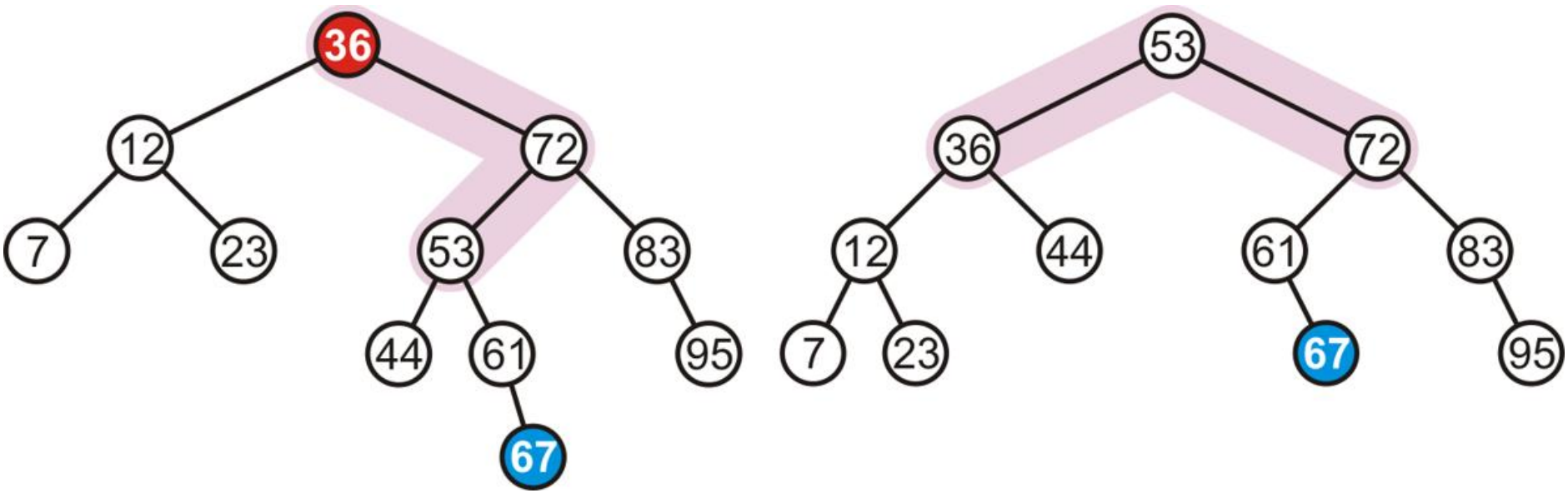


1. Perform a right rotation on the right subtree
2. Perform a left rotation on root

- Node $\mathbf{k_2}$ becomes the new root
- Node $\mathbf{k_2}$ takes ownership of node $\mathbf{k_3}$ as its right child
- Node $\mathbf{k_1}$ takes ownership of node $\mathbf{k_2}$'s left child as its right child
- Node $\mathbf{k_3}$ takes ownership of node $\mathbf{k_2}$'s right child as its left child

Right-Left Rotation (RL) or "Double Left" – Example

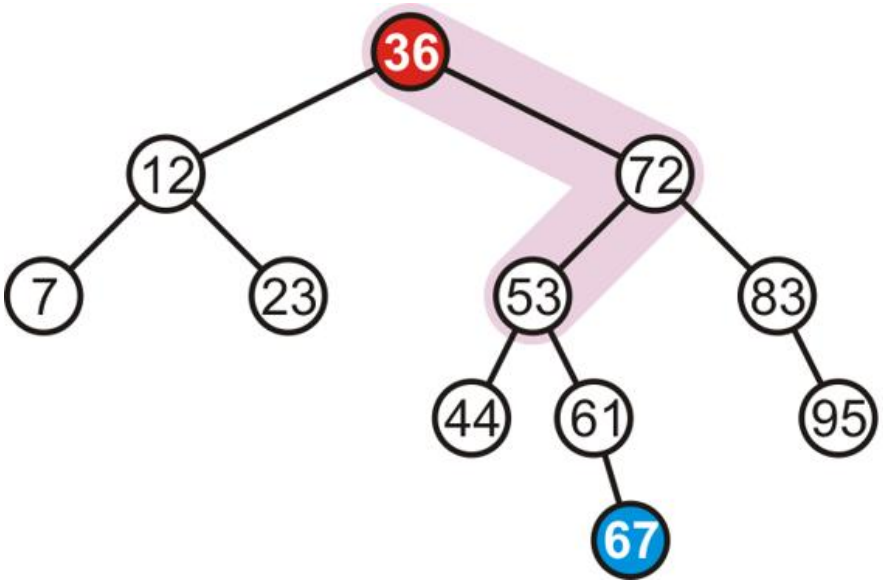
- Consider adding 67
  - To fix the imbalance, we perform Right-Left (RL) rotation of root



When to Perform Right-Left Rotation (RL)

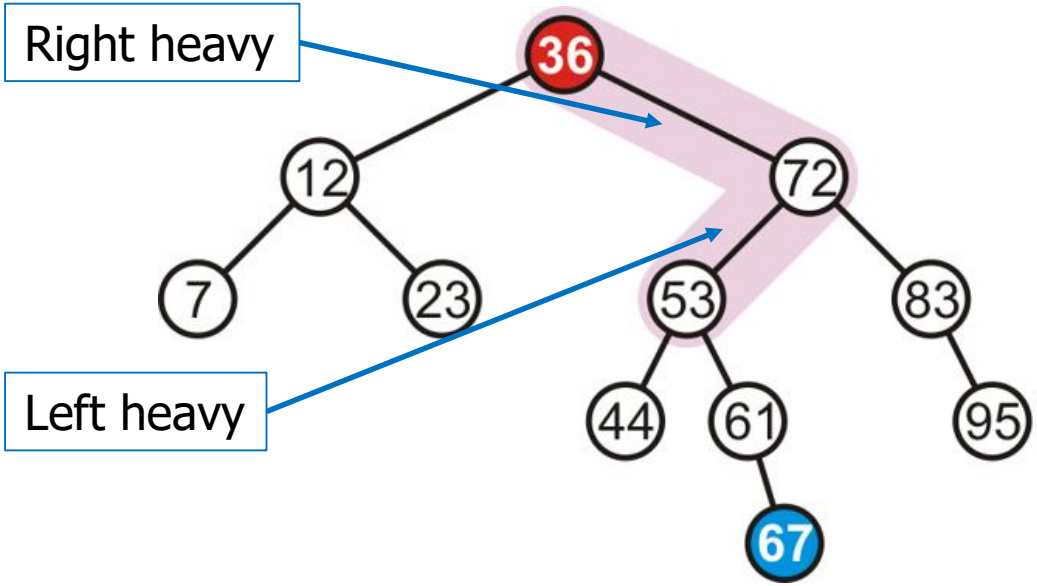
- Let the node that needs rebalancing be a

- Case LR
  - Insertion into left subtree of right child of node a



When to Perform Right-Left Rotation (RL)

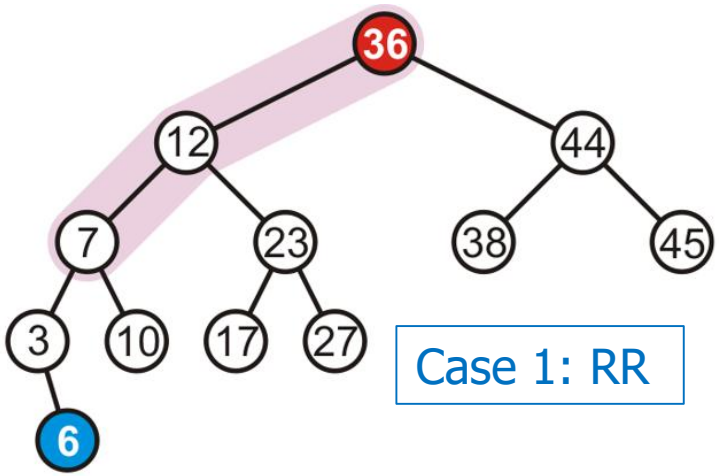
- Let the node that needs rebalancing be a
- Case LR
  - Insertion into left subtree of right child of node a (**LR**)



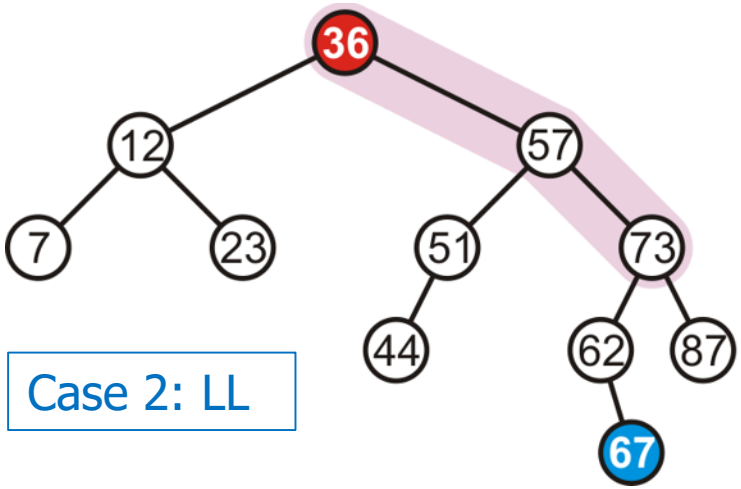
Summary: How And When To Rotate?

- Let the node that needs rebalancing be "a"
- Violation during insertion may occur in four cases

- Outside cases (Single Rotation)
  1. Insertion into left subtree of left child of node a (case RR)



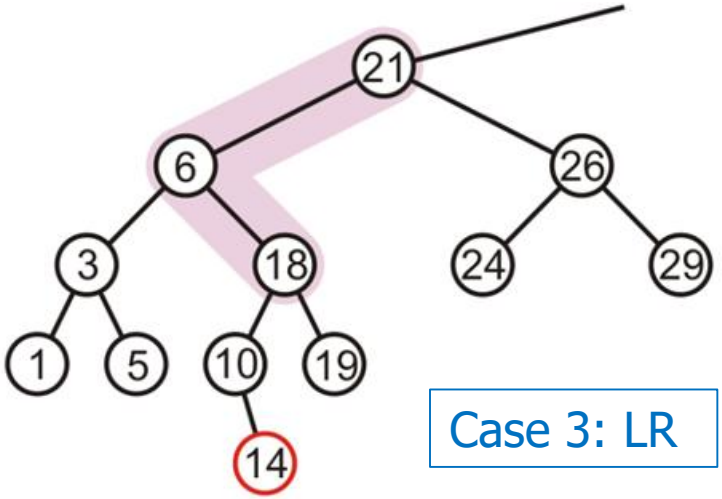
  2. Insertion into right subtree of right child of node a (case LL)



Summary: How And When To Rotate?

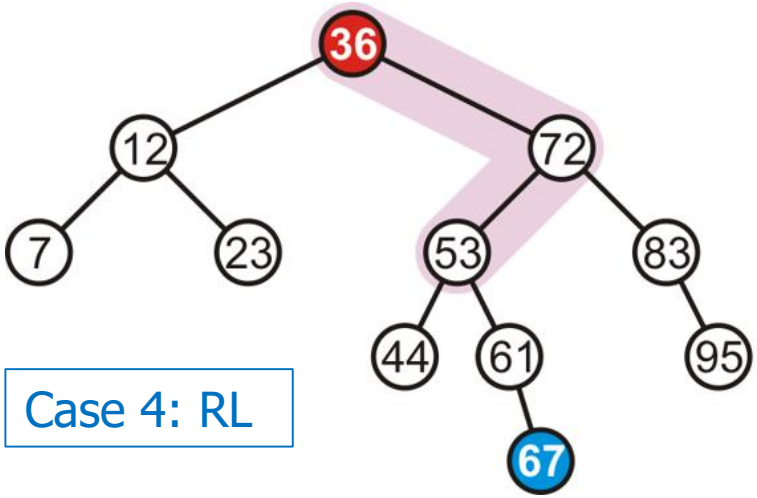
- Let the node that needs rebalancing be "a"
- Violation during insertion may occur in four cases

- Inside cases (Double Rotation)
  3. Insertion into right subtree of left child of a (case LR)



Case 3: LR

  4. Insertion into left subtree of right child of a (case RL)



Case 4: RL

Summary: How And When To Rotate?

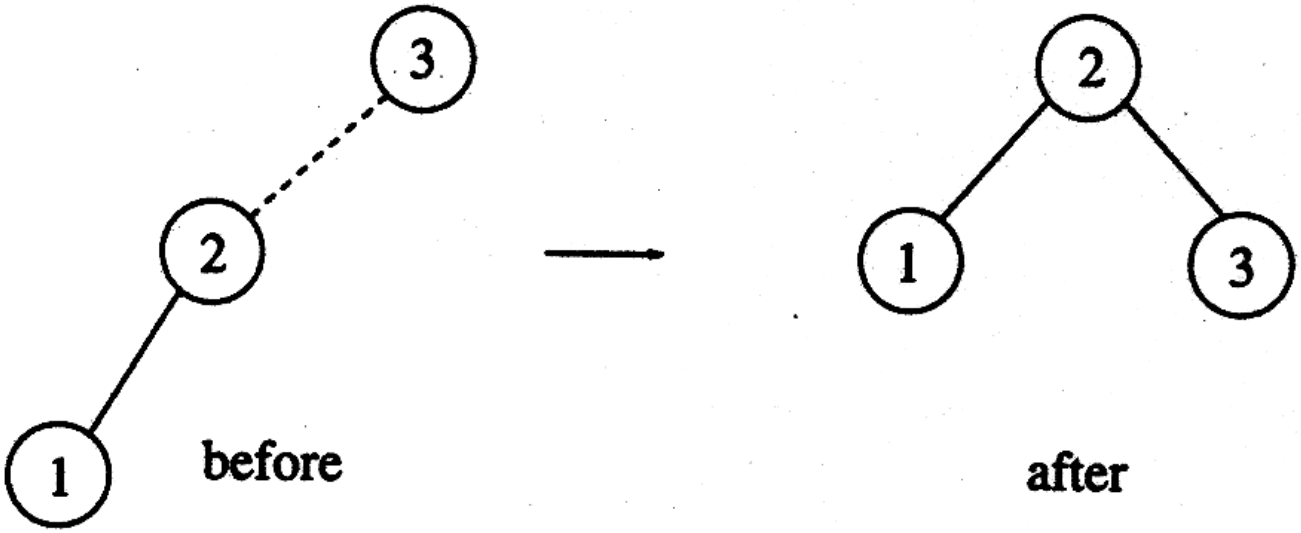
```
if tree is right heavy {
  if tree's right subtree is left heavy {
     Perform Right-Left rotation
  }
  else {
     Perform Single Left rotation
  }
}
…
```

```
…
else if tree is left heavy {
  if tree's left subtree is right heavy {
     Perform Left-Right rotation
  }
  else {
     Perform Single Right rotation
  }
}
```

AVL Tree – Complete Example

- Construct AVL Tree with the following input elements
  - 3, 2, 1, 4, 5, 6, 7

Insert 3, 2, 1

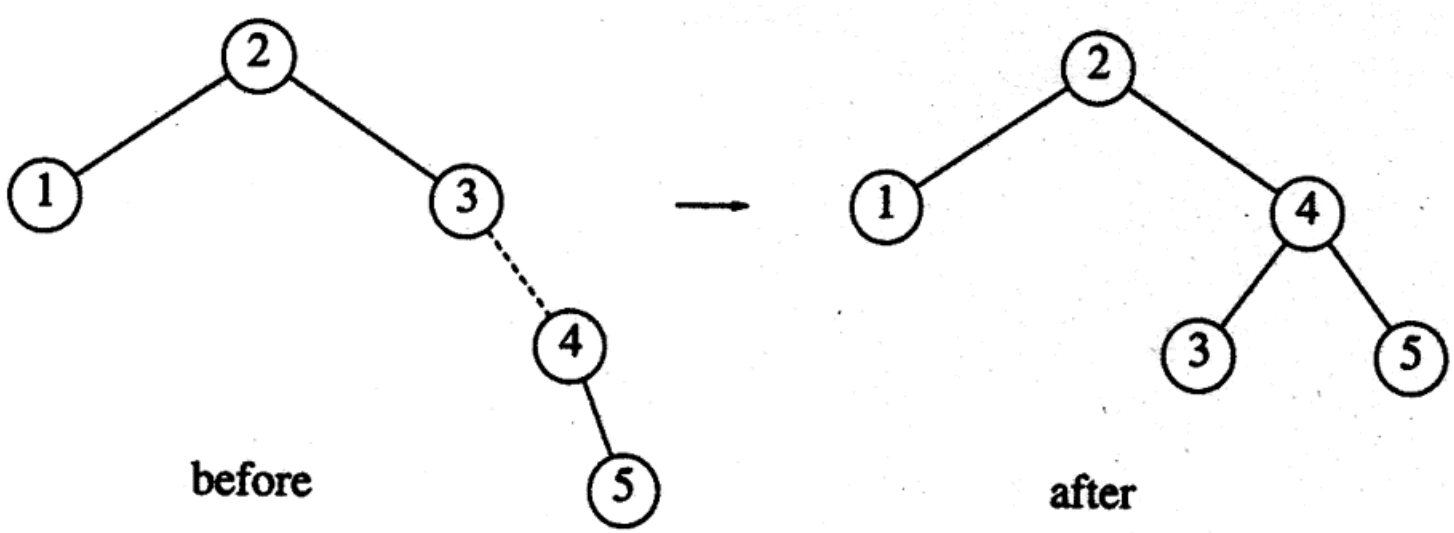


Insert 4, 5

AVL Tree – Complete Example

- Construct AVL Tree with the following input elements
  - 3, 2, 1, 4, 5, 6, 7

Insert 4, 5

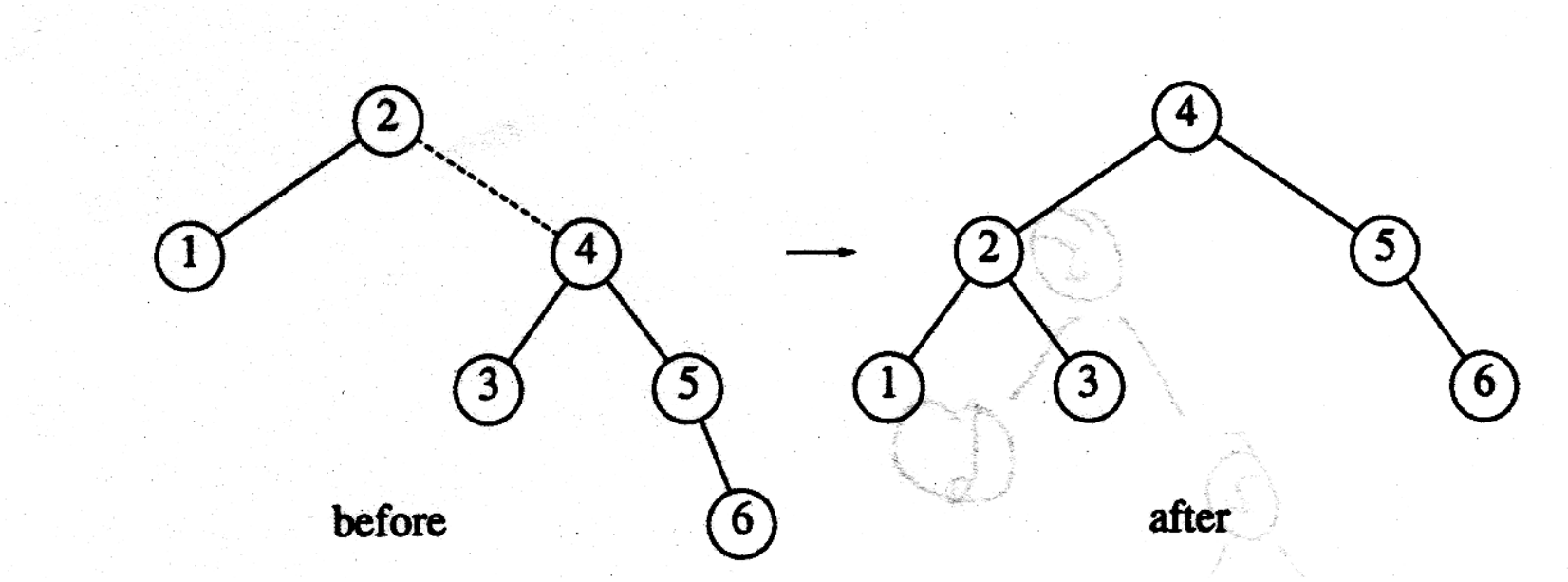


Insert 6

AVL Tree – Complete Example

- Construct AVL Tree with the following input elements
  - 3, 2, 1, 4, 5, 6, 7

Insert 6

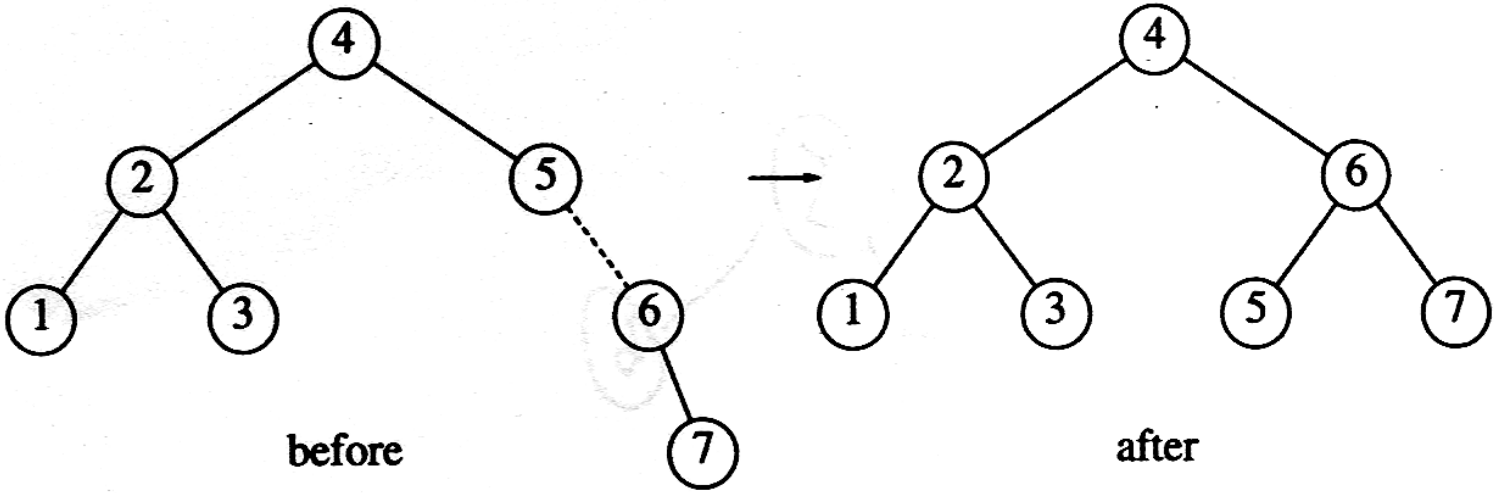


Insert 7

AVL Tree – Complete Example

- Construct AVL Tree with the following input elements
  - 3, 2, 1, 4, 5, 6, 7

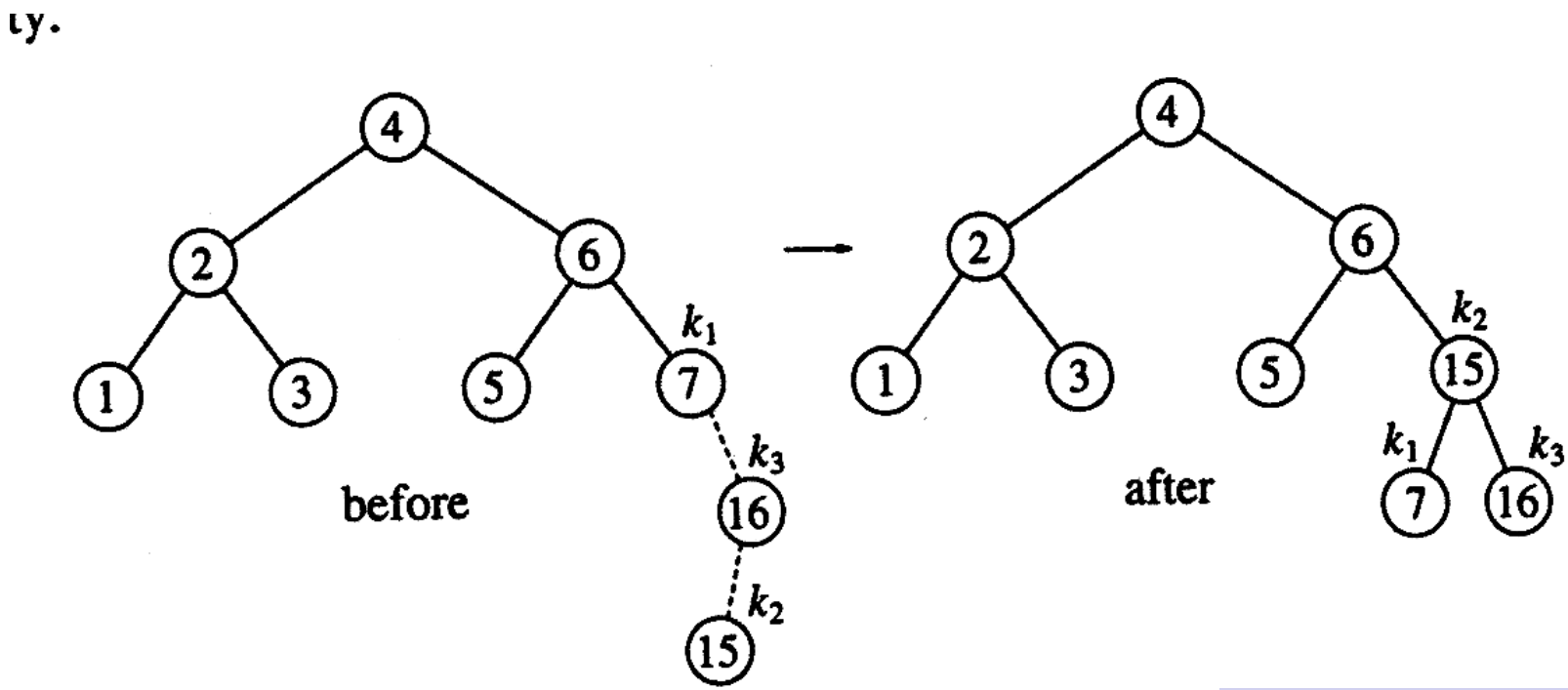
Insert 7



AVL Tree – Complete Example

- Suppose the following elements have to be inserted further
  - 16, 15, 14, 13, 12, 11, 10, 8

Insert 16, 15

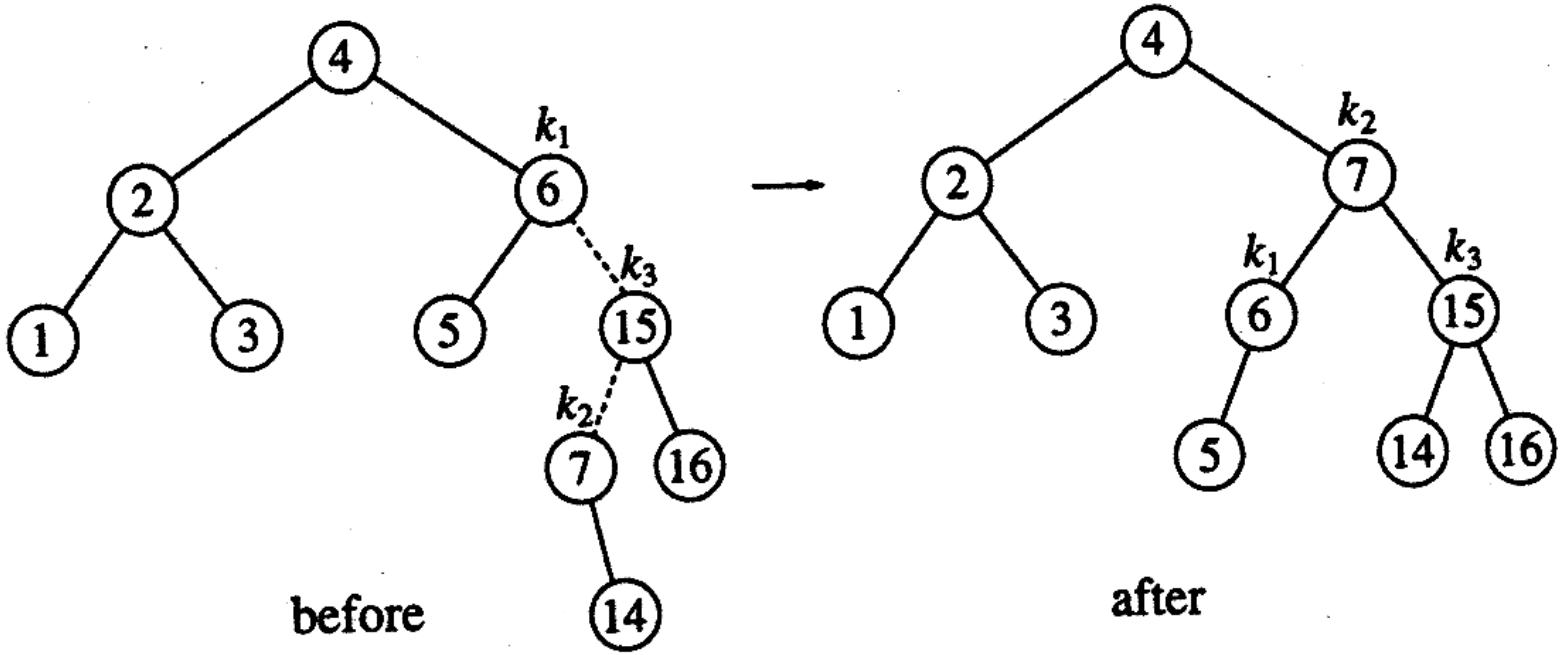


Insert 14

AVL Tree – Complete Example

- Suppose the following elements have to be inserted further
  - 16, 15, 14, 13, 12, 11, 10, 8

Insert 14

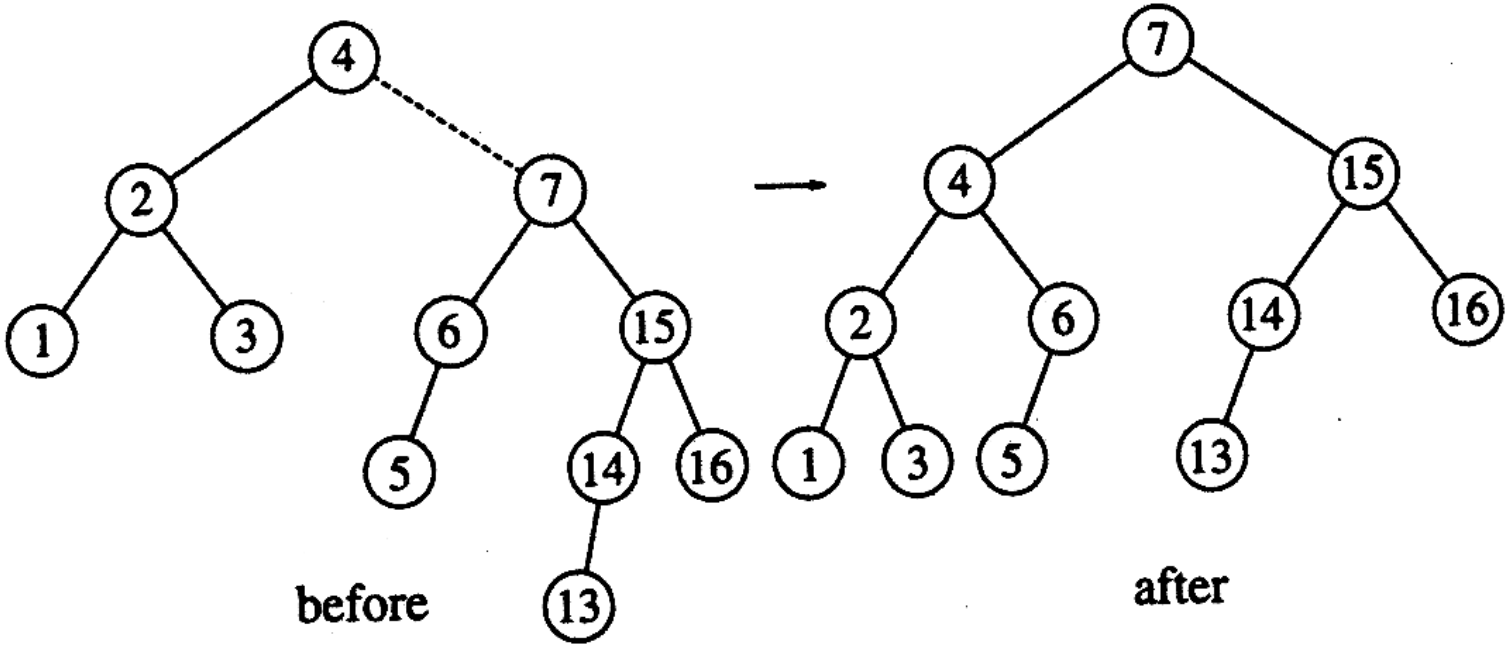


Insert 13

AVL Tree – Complete Example

- Suppose the following elements have to be inserted further
  - 16, 15, 14, 13, 12, 11, 10, 8

Insert 13

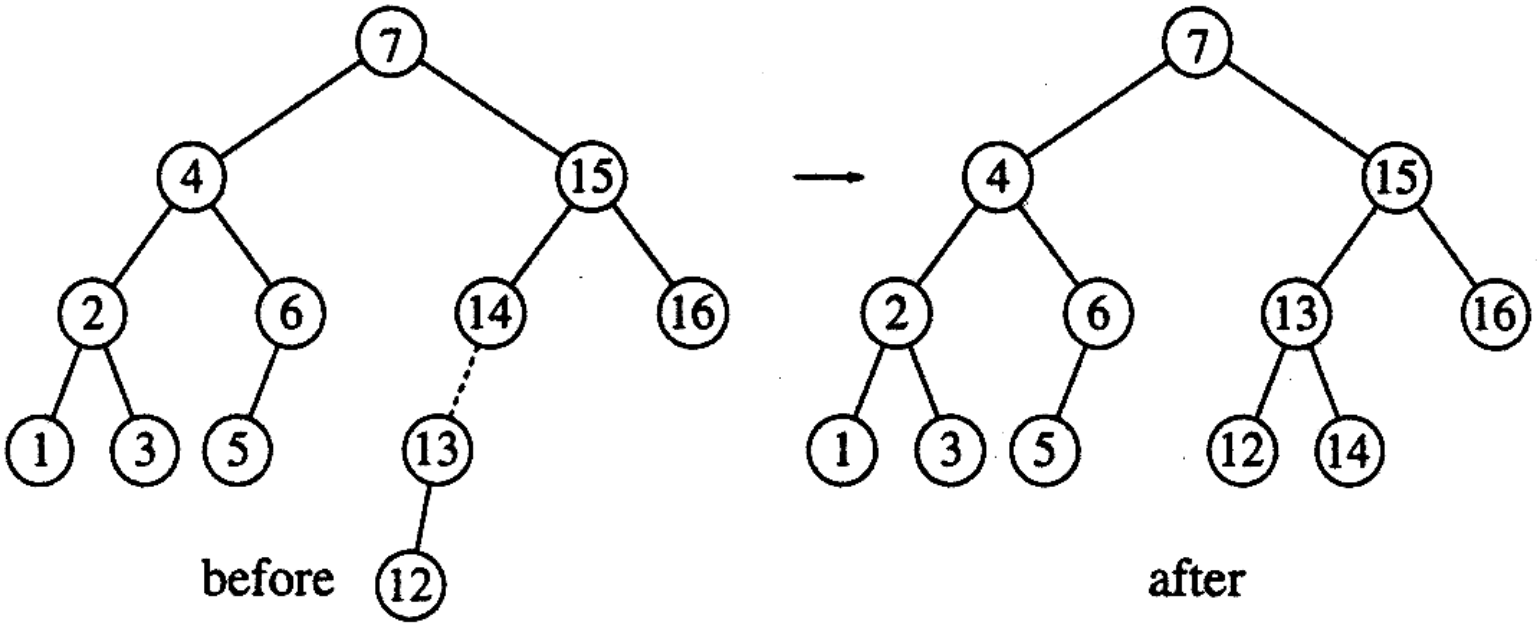


Insert 12

AVL Tree – Complete Example

- Suppose the following elements have to be inserted further
  - 16, 15, 14, 13, 12, 11, 10, 8

Insert 12

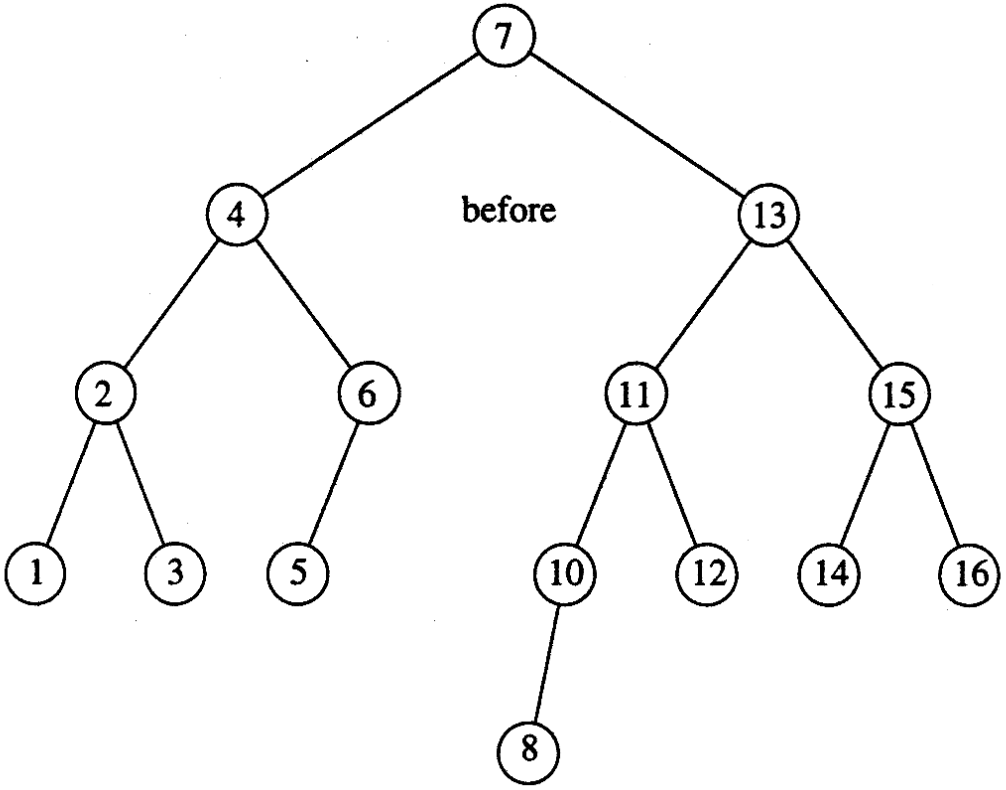


Insert 11, 10

AVL Tree – Complete Example

- Suppose the following elements have to be inserted further
  - 16, 15, 14, 13, 12, 11, 10, 8

Insert 11, 10 then 8



Any Question So Far?